

Tackling the Data-Parallel Load Balancing Bottleneck in LLM Serving: Practical Online Routing at Scale

Tianci Bu^{1*} Yuan Lyu^{2*} Zixi Chen^{3†} Chendong Song^{1†} Hong Liang²
Tsepen Gurung¹ Yuwei Fan^{2‡} Yinyu Ye^{1‡} Zijie Zhou^{1‡}

¹Department of Industrial Engineering and Decision Analytics, HKUST

²Theory Laboratory, Central Research Institute, 2012 Laboratories, Huawei

³School of Mathematical Sciences, Peking University

{tcbu, songcd, yyye, jerryzhou}@ust.hk

{lyu.yuan, liang.hong3, fanyuwei2}@huawei.com
chenzixi22@stu.pku.edu.cn

Abstract

Data-parallel (DP) load balancing has emerged as a first-order bottleneck in large-scale LLM serving. When a model is sharded across devices via tensor parallelism (TP) or expert parallelism (EP) and replicated across many DP workers, every decode step ends in a synchronization barrier whose latency is set by the most heavily loaded worker; even modest persistent imbalance across DP workers compounds, step after step, into a substantial fraction of wasted compute. The problem is hard for reasons specific to LLM decoding: assignments are sticky (migrating KV caches has a high cost), per-request loads grow over time, arrivals are non-stationary, and the router must decide within a sub-100 ms decode budget over hundreds of waiting requests and tens of workers. We present **BalanceRoute**, a family of practical online routing algorithms that target this bottleneck. The first, **BR-0**, requires no prediction infrastructure and uses a piecewise-linear F-score that captures the sharp asymmetry between admissions that fill safe margin and those that overflow into the envelope; a two-stage decomposition keeps per-step cost compatible with millisecond-scale scheduling. The second, **BR-H**, generalizes BR-0 with a short, constant lookahead H and a lightweight termination-classifier interface, extending the F-score to a horizon-discounted form. We deploy BalanceRoute on a 144-NPU cluster and evaluate against vLLM baselines on both a proprietary production trace and the public Azure-2024 trace. Across both workloads, BalanceRoute substantially reduces average DP imbalance and improves end-to-end serving throughput.

1 Introduction

Modern large language models (LLMs) are too large to fit on a single accelerator, so production serving systems shard each model across the devices within a worker group via tensor parallelism or expert parallelism (TP/EP) [Shoeybi et al., 2019, Fedus et al., 2022, Liu et al., 2024], and replicate the resulting group across many parallel workers via data parallelism (DP) to scale throughput [Kwon et al., 2023]. Within-group sharding creates collective communication (TP all-reduce or EP all-to-all)

*Equal contribution.

†Equal contribution.

‡Corresponding authors.

at every decode step, imposing a synchronization barrier across all DP replicas vLLM Contributors [2026]; each decode step ends only when the slowest replica is done. At large scale, this barrier turns cross-replica load imbalance into a dominant source of inefficiency. By a standard order-statistics argument, the gap between the maximum and the mean per-worker load grows with the number of workers, so the idle fraction of every decode step worsens as the system scales up; Chen et al. [2026] reports that more than 40% of accelerator time is lost to barrier idle in production LLM deployments. The cost compounds beyond throughput: idle accelerators continue to draw substantial power [Özcan et al., 2025], converting wasted compute into wasted energy. DP load balancing has therefore become a first-order bottleneck on industrial LLM serving efficiency.

Despite this importance, DP load balancing in LLM serving is poorly served by the heuristics deployed in practice. Production routers in open-source serving stacks [vLLM Project, 2025] rely on generic load-balancing heuristics: round-robin, join-shortest-queue, random, power-of-two-choices [Mitzenmacher, 2002]. These heuristics are agnostic to the features distinguishing LLM decoding from classical parallel computing: sticky assignments tied to per-request KV caches, per-step load that grows with each generated token, and routing decisions made at sub-100 ms decode latency over hundreds of waiting requests and tens to hundreds of replicas (more backgrounds and details are provided in Section 2). Zhou [2026] identifies this DP-decode routing problem as a representative case where the gap between generic load-balancing heuristics and LLM-specific structure calls for principled algorithm design. In addition, Chen et al. [2026] recently formalized this problem by an analytical model along with theoretical insights based on a short-horizon lookahead integer optimization formulation. Building on their insights, we focus on a complementary algorithmic question: *how to design routers that achieve strong DP load balance at real serving scale within the millisecond per-step budget faced by production deployments*. Other related work is deferred to Appendix A.

In this work, we present *BalanceRoute*, a family of practical online DP load-balancing routers, and validate them at production scale. Our contributions are:

- (i) **A prediction-free online router (BR-0, Section 3).** We derive a piecewise-linear F-score from the marginal effect of an admission on single-step imbalance and use it to drive a two-stage algorithm.
- (ii) **A lookahead-aware online router (BR-H, Section 4).** When a lightweight termination classifier is available, BR-H extends the F-score to a constant-horizon discounted form within the same two-stage structure. BR-0 emerges as the $H=0$, so the two algorithms form a single family.
- (iii) **Production-scale empirical validation (Section 6).** We deploy BalanceRoute on a 144-NPU Ascend 910C cluster and evaluate it against vLLM baselines on a proprietary production trace and the public Azure-2024 trace. We observe substantial reductions in average DP imbalance and corresponding throughput gains, with the gains growing in the regimes our analysis predicts.

2 Background and Problem Formulation

This section describes the data-parallel (DP) decode setting in which our routers operate, identifies the structural reasons why DP load balancing is hard, and introduces the formal model.

2.1 DP-decode load imbalance

Why DP is the relevant unit of imbalance. Modern LLMs are too large for a single worker to host a full model replica. Production serving systems shard parameters across devices using tensor parallelism (TP) or, for mixture-of-experts models, expert parallelism (EP), and replicate the resulting model across multiple groups using data parallelism. Each DP group serves an independent batch of decoding requests. Load imbalance *across* DP workers is determined entirely by how the router assigns requests, and is the bottleneck we target.

Sticky assignments and growing KV caches. We focus on the router which sends completed prefill requests to decode workers. Under prefill-decode (PD) disaggregation Zhong et al. [2024], Patel et al. [2023], prompts are processed in a separate prefill cluster; once a request finishes prefill, it is routed to a decode worker which then generates output tokens autoregressively. Each decoded token appends one entry to the request’s key-value (KV) cache, so the per-request memory and attention cost grow monotonically over the lifetime of the request. Migrating an in-flight request would require

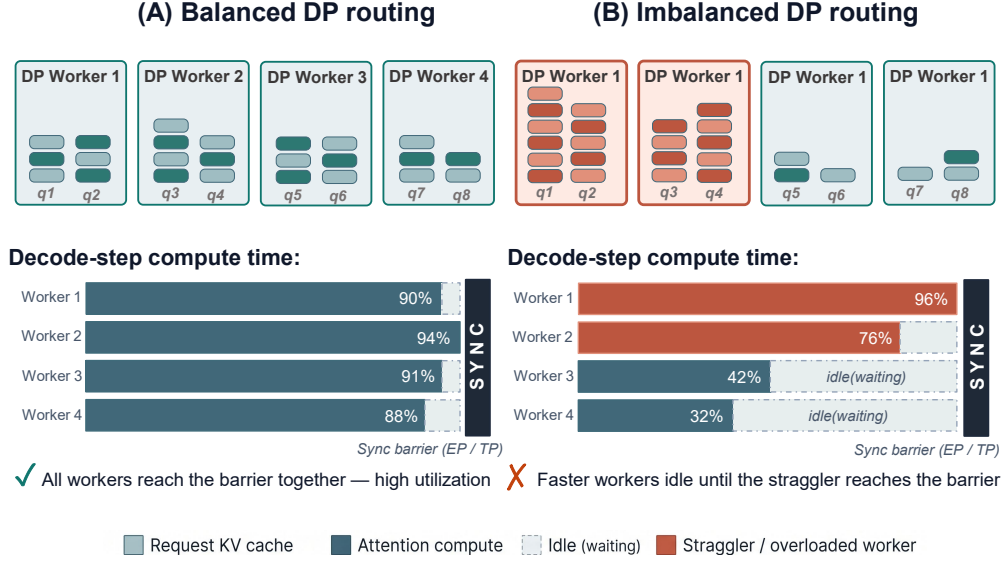


Figure 1: Data-parallel decode under barrier synchronization.

transferring its full KV cache across devices at every step; in practice, therefore, assignments are *sticky*: once a request lands on a worker, it remains there until completion.

Barrier synchronization. The bulk of per-step compute on a decode worker is to read each active request’s KV cache; the per-step load on a worker is therefore approximately linear in the total KV-cache footprint of its active batch. Within a single decode step, the workers in a DP fleet do not run independently. Each step contains collective communication—EP all-to-all when the model uses MoE layers Liu et al. [2024], TP all-reduce when attention or MLP weights are sharded—that imposes a step-wise synchronization barrier across all DP workers vLLM Contributors [2026]. Concretely, every worker must finish its per-step compute before any worker can proceed, and step latency is set by the slowest participant. Figure 1 contrasts a balanced and an imbalanced assignment under this constraint: in the balanced case, all workers reach the barrier together, whereas in the imbalanced case the lighter workers spend a substantial fraction of every step waiting for the straggler. Because the per-worker idle accumulates at *every* decode step, even a modest persistent imbalance compounds into a large fraction of wasted compute over the lifetime of a request.

Why is DP load-balancing problem hard? Four properties together place the DP-decode setting outside the regime in which classical online makespan or queueing-based load balancing applies: (i) *Sticky assignment*. Decisions cannot be revised: migrating a request has a huge cost since we need to carry all its KVs. (ii) *Unknown, dynamic per-request workload*. The remaining output length of a request is not known at routing time, and the per-step attention cost grows as the KV cache grows. The router must therefore reason about not just the current load but the evolving load implied by its assignments. (iii) *Non-stationary arrivals*. Requests arrive from prefill in bursts whose composition is workload-dependent and shifts over time. (iv) *Millisecond decision budget*. The router runs at every decode step (sub-100 ms in production). This budget rules out per-step solutions to decision rules with heavy computations.

When does DP load imbalance matter most? Two regimes amplify the impact of DP-decode imbalance and explain why this problem has become a first-order concern only at modern serving scale: (i) **Large scale LLM inference**. As the number of DP workers G grows, by a standard order-statistics argument, the gap between the maximum load and the mean load grows with G . Barrier idle scales with this gap, so the load-balancing penalty is super-linear in G . This is the regime in which industrial serving operates and in which our large-scale experiments (Section 6) reside. (ii) **Bandwidth-driven per-step cost**. The per-step time on a worker is approximately $ax + b$, where x is the total KV-cache footprint of the worker’s active batch, a is the bandwidth, and b is a load-independent overhead. DP load balancing is therefore most consequential when the bandwidth-driven

term ax is large relative to the fixed overhead b —the regime of long contexts, high effective batch sizes, or hardware with relatively low bandwidth-to-overhead ratios.

2.2 Mathematical model

We now formalize the prefill-to-decode routing problem. Let G be the number of DP workers, indexed by $g \in [G]$, each with a maximum concurrency of $B \in \mathbb{N}$ active requests. Decode step is discrete and indexed by $k \in \{0, 1, 2, \dots\}$: at step k , every active request on every worker advances by one decode iteration, after which the workers synchronize at the barrier described above.

Requests and workload profiles. Let $\mathcal{I}$ be the set of all requests in a given trace. Each request $i \in \mathcal{I}$ becomes available to the router at some step k_i (its prefill-completion time) and is assigned to a worker at some step $x_i \geq k_i$. Once assigned, i is processed continuously on that worker until completion; assignments are sticky and we never preempt. The processing requirement of i is summarized by a *workload profile* $W_i = (w_i^{(1)}, w_i^{(2)}, \dots, w_i^{(o_i)})$, where $o_i \in \mathbb{N}$ is the total number of decode steps performed for i , and $w_i^{(j)} \geq 0$ is the KV workload contributed by i during its j -th decode step on the assigned worker. Specifically, a request i whose prefill produced s_i tokens has $w_i^{(j)} = s_i + j - 1$. The profile W_i is partially unobserved by the router at assignment time: the router knows i has arrived and the value of $w_i^{(1)} = s_i$ (prefill size) but does not know how long it will run (o_i is unknown).

Per-step state. Let $g(i)$ denote the worker to which request i is assigned. For each worker g and step k , define the active set $\mathcal{A}_g(k) = \{i \in \mathcal{I} : g(i) = g, x_i \leq k < x_i + o_i\}$, i.e., the requests currently being processed on worker g at step k . The instantaneous load on worker g at step k is $L_g(k) = \sum_{i \in \mathcal{A}_g(k)} w_i^{(k-x_i+1)}$, the sum of step- $(k - x_i + 1)$ workloads of all requests currently active on g . Let $R_{\text{wait}}(k)$ denote the *waiting set* of requests that have completed prefill by step k but have not yet been assigned to a decode worker.

Routing policy. A non-anticipative routing policy π specifies, at each step k , an assignment $\{S_g(k)\}_{g \in [G]}$ where $S_g(k) \subseteq R_{\text{wait}}(k)$ is the (possibly empty) set of waiting requests dispatched to worker g at step k , subject to the capacity constraint $|\mathcal{A}_g(k)| + |S_g(k)| \leq B$, for all $g \in [G]$, and to the disjointness constraint $S_g(k) \cap S_{g'}(k) = \emptyset$ for $g \neq g'$. The decision at step k may depend only on observable state up to step k ; in particular, it cannot depend on the latent profiles $\{W_i\}$.

3 BR-0: Prediction-Free Balance Routing

We begin with the first router, *BalanceRoute-0* (BR-0), where no decode length prediction information is needed. At step k , the scheduler observes only the current state $\{\mathcal{A}_g(k), L_g(k)\}_{g \in [G]}$ and the waiting set $R_{\text{wait}}(k)$. For request i , write $s_i := w_i^{(1)}$ for its load at admission (the prefill length). When the router admits a subset $Q \subseteq R_{\text{wait}}(k)$ to worker g at step k , the immediate increment to $L_g(k)$ is $\Delta_s(Q) := \sum_{i \in Q} s_i$.

3.1 Marginal current-step imbalance

We define the total imbalance at decode step k as $I(k) = G \cdot M(k) - \sum_{g \in [G]} L_g(k)$, where $M(k) := \max_{g \in [G]} L_g(k)$, i.e., the work that lighter workers would have done if loaded as heavily as the heaviest. Routing decisions at step k change $\{L_g(k)\}$ for admitted requests but not the active sets of already-running ones; the policy’s lever on $I(k)$ is therefore the choice of admissions $\{S_g(k)\}_{g \in [G]}$.

For a candidate subset $Q \subseteq R_{\text{wait}}(k)$ and worker g , let $m_g := M(k) - L_g(k) \geq 0$ denote the *safe margin*: the additional load g can absorb before overtaking the current heaviest worker. Admitting Q to worker g changes $L_g(k)$ to $L_g(k) + \Delta_s(Q)$ and $M(k)$ to $\max(M(k), L_g(k) + \Delta_s(Q))$, so the change in $I(k)$ is $\Delta I_g(Q) = G \cdot (\Delta_s(Q) - m_g)_+ - \Delta_s(Q)$. We define a per-worker metric, *F-score*, which measures the imbalance reduction induced by admitting Q to g :

$$F_g(Q) := -\Delta I_g(Q) = \Delta_s(Q) - G \cdot (\Delta_s(Q) - m_g)_+. \quad (1)$$

Equation (1) is piecewise linear in $\Delta_s(Q)$ with two regimes:

- **Safe** ($\Delta_s(Q) \leq m_g$). The score is purely linear, $F_g(Q) = \Delta_s(Q)$; placing more load on g strictly reduces $I(k)$ as long as g does not overtake the current heaviest worker.
- **Overflow** ($\Delta_s(Q) > m_g$). The score becomes $F_g(Q) = G m_g - (G - 1) \Delta_s(Q)$; once g overtakes the previous heaviest worker, every additional unit of load on g raises $M(k)$ and *increases* imbalance by a factor of $G - 1$.

The crossover at $\Delta_s(Q) = m_g$ is sharp: pushing slightly above m_g flips a 1-per-unit benefit into a $(G-1)$ -per-unit cost. At large scale, this asymmetry is severe, and a useful one-line summary of BR-0 is: *fill safe margins preferentially, and when overflow is unavoidable, route to the worker where it costs least.*

3.2 Two-stage algorithm

BR-0 splits the per-step decision into two stages, controlled by a threshold S_{greedy} on the total free capacity $S_{\text{tot}} := \sum_g \text{cap}[g]$, where $\text{cap}[g] := B - |\mathcal{A}_g(k)|$.

Stage 1: greedy fill (abundant capacity). While $S_{\text{tot}} > S_{\text{greedy}}$ and $R_{\text{wait}}(k)$ is nonempty, BR-0 selects the worker g with the largest free capacity and admits the single request $i \in R_{\text{wait}}(k)$ that maximizes $F_g(\{i\})$. Because g has the largest spare capacity, it is in the safe regime: $F_g(\{i\}) = s_i$ and we send the largest waiting request to the lightest-loaded worker with the most spare slots.

Stage 2: refined allocation (scarce capacity). Once $S_{\text{tot}} \leq S_{\text{greedy}}$, the few remaining slots have outsized leverage on $I(k)$, and crossing the safe-regime boundary becomes likely. BR-0 places workers with $\text{cap}[g] > 0$ into a priority queue keyed by $(\text{cap}[g], m_g)$, favoring workers with both larger free capacity and larger safe margin. For each worker g popped from the queue, BR-0 considers the head R_{max} candidates of $R_{\text{wait}}(k)$ (sorted by s_i) and selects the subset Q^* with $|Q^*| \leq \min(\text{cap}[g], R_{\text{max}})$ maximizing $F_g(Q)$. If the best subset has nonpositive score, BR-0 admits the single best request anyway to avoid starvation; the worker is reinserted into the queue if capacity remains. The full procedure is given in Algorithm 2 in Appendix B.

4 BR-H: Lookahead-Aware Balance Routing

In practice, predictions of each active request’s remaining decode length $r_i(k) := o_i - a_i(k)$ are often available at routing time [Zheng et al., 2023, Qiu et al., 2024, Fu et al., 2024, Shahout et al., 2024, Chen et al., 2025], where $a_i(k) := k - x_i$. A full prediction is, however, neither necessary nor accurate: Chen et al. [2026] show that a constant short lookahead H already controls $\sum_k I(k)$, and beyond the binary threshold “terminates within H steps,” the precise heavy-tailed value of $r_i(k)$ has only second-order effect on routing—so a binary classifier captures the routing-relevant signal more reliably than a full-length regressor. Even given exact $r_i(k)$, longer horizons $H' \gg H$ depend on unobservable future arrivals, whose noise grows faster than the signal. We therefore extend BR-0 with a constant H -step lookahead, yielding BR-H.

For a fix horizon H , we develop a *termination classifier* and *conditional-mean regressor* for predicting the workload in the future H decode steps. The detail of the short-horizon prediction interface can be found in Appendix C.1.

4.1 Horizon F-score

The F-score in BR-0 (1) measures the marginal effect of admitting Q to g on a single step. We extend it to the horizon by summing per-step marginal effects over $h = 0, \dots, H$, with two modifications: (i) a discount vector $\mathbf{d} := (1, \gamma, \dots, \gamma^H)^\top$ with $\gamma \in (0, 1]$ down-weights far-future imbalance, reflecting that near-term predictions are more reliable; and (ii) the binary penalty coefficient G in (1) is replaced by a tunable coefficient β , allowing empirical calibration of the overflow penalty under prediction noise. A scaling constant $\alpha > 0$ on the reward term plays the symmetric role for the safe regime.

Approximating each request in Q as contributing a constant s_i of additional load over the horizon, the per-step margin consumed by Q at step $k + h$ is $(\Delta_s(Q) - m_{g,h})_+$, while the per-step reduction in total load deficit is $\Delta_s(Q)$ whenever g remains under the envelope. Aggregating with $\mathbf{d}$ gives the horizon F-score

$$F_g(Q) = \alpha (\mathbf{1}^\top \mathbf{d}) \Delta_s(Q) - \beta (\Delta_s(Q) \mathbf{1} - \mathbf{m}_g)_+^\top \mathbf{d}, \quad (2)$$

with tunable parameters (α, β, γ) . As in BR-0, $F_g(Q)$ is piecewise linear in $\Delta_s(Q)$ with two regimes: *horizon-safe* ($\Delta_s(Q) \leq \min_h m_{g,h}$), where the penalty term vanishes and $F_g(Q)$ is linear in $\Delta_s(Q)$; and *overflow* ($\Delta_s(Q) > \min_h m_{g,h}$), where the penalty activates step by step as the safe envelope is exceeded. A smaller γ trusts predictions less and pulls BR-H closer to BR-0’s myopic behavior.

Reduction to BR-0. Substituting $H = 0$ into (2) yields $\mathbf{d} = (1)$, $\mathbf{m}_g = (M(k) - L_g(k))$, which coincides with the BR-0 score (1) when $(\alpha, \beta) = (1, G)$.

4.2 Two-stage algorithm

BR-H follows the same two-stage structure as BR-0, with three modifications to use horizon information: (i) scoring uses the horizon F-score (2) instead of (1), (ii) the priority-queue key in Stage 2 uses the horizon-min margin $\min_h m_{g,h}$ rather than the single-step margin, and (iii) once per scheduling round, BR-H computes the horizon projection $\{L_g(k+h)\}_{h=0}^H$, $\mathbf{M}$, and $\{\mathbf{m}_g\}$ from the predicted profiles, with subsequent admissions updating these incrementally. The full procedure is given in Algorithm 3 in Appendix B.

Algorithm 1: BalanceRoute-H (BR-H) (Intuition). Full algorithm with predictor refresh, tie-breaking, starvation guard, and state-update bookkeeping in Algorithm 3 in Appendix B.

Input: Waiting pool R_{wait} , active sets $\{\mathcal{A}_g\}$, horizon H , F-score F_g (2)

Output: Per-worker admissions at step k

Compute horizon-projected margins $\{\mathbf{m}_g\}$ from cached predictions $\{\hat{c}_i\}$;

Stage 1 — abundant capacity: while total free slots $> S_{\text{greedy}}$, pick the worker g with the most free slots and admit the single request $i^* = \arg \max_{i \in R_{\text{wait}}} F_g(\{i\})$;

Stage 2 — scarce capacity: for each remaining worker, in priority order $(\text{cap}_g, \min_h m_{g,h})$, admit subset $Q^* \subseteq R_{\text{wait}}$ with $|Q^*| \leq R_{\text{max}}$ maximizing $F_g(Q)$;

5 System Implementation

We deploy BR-H on vllm-ascend [vLLM Community, 2025], the Ascend-NPU port of vLLM [Kwon et al., 2023], in a prefill–decode (PD) disaggregated configuration, validated on Qwen3-30B-A3B (MoE) on Atlas 910B and DeepSeek-V3 W8A8 on Atlas 910C (full topologies in Section 6). The BR-H rule of Section 4 runs in milliseconds once the cluster snapshot

$$\left(\{\mathcal{A}_g(k), L_g(k)\}_{g \in [G]}, R_{\text{wait}}(k), \{\hat{c}_i(k)\}_{i \in \cup_g \mathcal{A}_g(k)} \right) \quad (3)$$

is available at every decode step; the engineering task is to maintain it under non-preemptive cross-tier KV transfer, no engine introspection, and a sub-100 ms decision window.

5.1 Architecture and design principles

Three principles shape the design. *Centralized state:* the horizon F-score (2) requires a cross-worker view of projected loads and Stage 2 selects subsets from a pool, so a stateless router cannot serve (3); we therefore place a single stateful Python proxy in front of unmodified vllm-ascend workers. *Above-the-engine integration:* we drive the prefill–decode lifecycle through payload mutation on the standard OpenAI-compatible API and recover per-token decode progress by inline parsing of Server-Sent Events (SSE) streams, requiring no engine-binary modification. *Pool decoupling:* prefill-completed requests buffer in a *prompt pool* until the decode dispatcher wakes with a global view, enabling Stage 2 subset selection at the cost of late-binding KV transfer—bounded empirically, since deployed BR-H still beats every baseline on TPOT-P95 (Table 1).

The system has three tiers (Figure 2): a prefill tier and a decode tier of vllm-ascend instances, with the stateful proxy (Python/uvloop) between client and engines; cross-tier KV transfer uses MooncakeConnector [Qin et al., 2025]. The proxy maintains snapshot (3) in process memory through per-worker DecodeInstanceState[g] mirroring the executor’s active batch (with $L_g(k)$ derived in $O(1)$), a PromptPool supplying $R_{\text{wait}}(k)$, and a PrefillerBudget[i] caching KV-utilization samples from each prefiller’s Prometheus endpoint as back-pressure. Event-driven coroutines update the structures of prefill dispatch, decode dispatch, telemetry polling, and pool reaping;

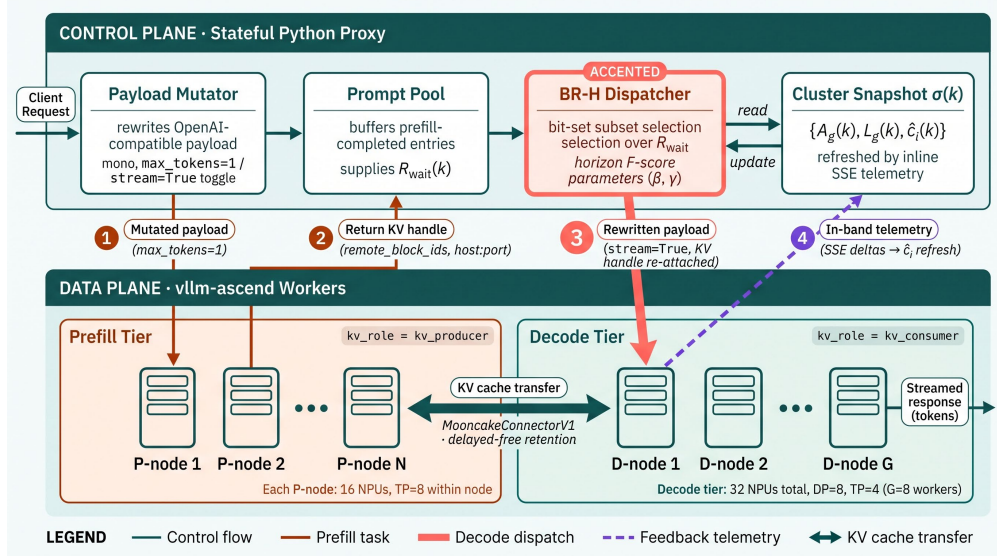


Figure 2: **System architecture.** A stateful proxy maintains the cluster snapshot and runs the BR-H rule. It drives the prefill tier (kv_role=kv_producer) via payload mutation, buffers completed requests in a *Prompt Pool*, and then dispatches them in batches to the decode tier (kv_role=kv_consumer). Load projections are rectified by monitoring per-token SSE telemetry from the decoders.

the payload-mutation contract, retry/cancellation logic, a latency-optimized BR-0 pool-bypass path, and how to solve the BR-H rule in milliseconds are detailed in Appendix D.

6 Experiments

6.1 Experimental setup

Serving stack. All experiments use the deployment of Section 5: vllm-ascend [vLLM Community, 2025] with V1 scheduler, continuous batching, PagedAttention [Kwon et al., 2023], and cudagraph_mode=FULL_DECODE_ONLY on the decode tier; MooncakeConnector [Qin et al., 2025] handles cross-tier KV transfer. Our 144-NPU cluster runs DeepSeek-V3 671B (W8A8) on Atlas 910C in a 4P1D topology by default: four prefill nodes (16 NPUs each, TP= 8) and one decode node (32 NPUs, DP= 8, TP= 4), giving $G=8$ decode workers; batch limit $B = \max_num_seqs$.

Datasets. We evaluate on two prefill–decode traces with markedly different output-length distributions, both run at heavy load (cluster near saturation). **Proprietary Data** is a proprietary production trace with heavy-tailed outputs (8,000 requests, mean prompt 3,197, mean output 1,185). **Azure-2024** [Microsoft Azure Project, 2025, Stojkovic et al., 2025] is the conversation split of the public Azure LLM Inference Trace, filtered to output length > 1000 tokens—the regime in which decode-stage imbalance dominates and outputs are cap-bounded (10,000 requests, mean prompt 4,652, mean output 1,052).

Baselines. We compare against four load-balancing policies from the vLLM router [vLLM Project, 2025]: **Random**, **Round-Robin (RR)**, **Power-of-Two-Choices (P2C)** [Mitzenmacher, 2002], and **Join-Shortest-Queue (JSQ)**, the upstream vllm-ascend default. Workload-aware endpoint routers [Jain et al., 2024, 2025] target a structurally different setting (request-to-instance assignment without cross-replica barrier synchronization) and are not directly comparable; see Appendix A.

Our methods. We deploy two routers: **BR-0**, the prediction-free router of Section 3, and **BR-H** ($H=80$) of Section 4, in three configurations. *BR-H oracle* replaces $\hat{c}_i(k)$ with the trace ground-truth $\min(r_i(k), H)$ as a reference for perfect lookahead. *BR-H deployed* uses the two-stage estimator of Section C.1 with one of two realizations: **Survival** reads both stages off the marginal training-output

CDF, and **ExactMatch** maintains a prompt-hash-keyed CDF and falls back to Survival on key miss. Offline predictor accuracy on both workloads is in Appendix C.3.

Metrics. **Average imbalance** is the trace mean of per-step max – min KV workload across the G decode workers, the step-level proxy for barrier-induced idle. **TPOT P95** is the 95th-percentile time-per-output-token (ms), capturing decode-stage tail latency. **Throughput** is output tokens per second. We report decode-stage metrics only, since prefill is unaffected by decode-tier routing.

6.2 Main results on Proprietary Data and Azure-2024

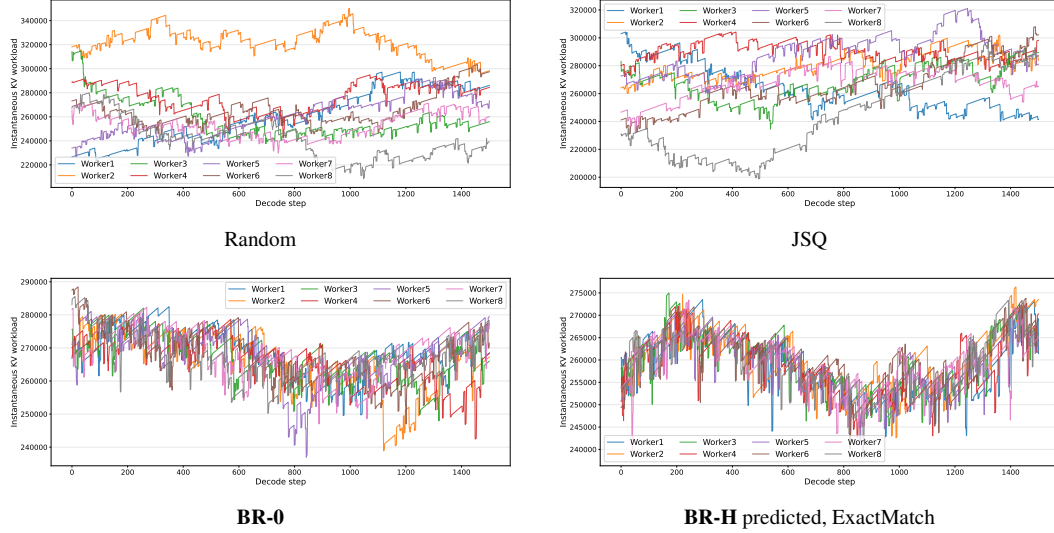


Figure 3: **Per-worker KV-cache workload on Proprietary Data, $G=8$.** Each panel plots all eight workers over the same 1,500-step decode segment; vertical spread is the instantaneous imbalance. The remaining panels (RR, P2C, BR-H oracle) are in Appendix F.

Table 1: **Main results on Proprietary Data and Azure-2024.** $G=8$, heavy load (Azure-2024 filtered to output length > 1000). Imbalance in tokens, TPOT P95 in ms, throughput (Tput) in tok/s; $\downarrow$ lower is better, $\uparrow$ higher is better. Primary deployment in **bold**.

Method	Proprietary Data			Azure-2024		
	Imbal $\downarrow$	TPOT $\downarrow$	Tput $\uparrow$	Imbal $\downarrow$	TPOT $\downarrow$	Tput $\uparrow$
Random	438,225	85.2	811	146,564	90.3	706
Round-Robin	254,798	83.2	847	110,049	89.1	771
Power-of-Two-Choices	262,940	84.0	836	109,033	89.3	794
Join-Shortest-Queue	215,110	83.2	843	104,737	89.3	785
BR-0 <i>prediction-free</i>	51,927	79.3	943	54,051	83.3	854
BR-H <i>oracle</i> , (43, 0.86)	23,975	78.0	1,022	37,831	78.1	951
BR-H <i>oracle</i> , (14.67, 0.64)	23,577	78.0	1,029	35,301	77.3	957
BR-H <i>deployed</i> , Survival	40,757	78.7	979	44,010	82.1	893
BR-H <i>deployed</i>, ExactMatch	36,277	78.8	977	38,496	82.3	909

Table 1 reports the heavy-load comparison at $G=8$. Figure 3 visualizes the load imbalance between different algorithms for the Proprietary Data trace, and Figure 8 in Appendix F visualizes the one for the Azure-2024 trace. The qualitative ordering is identical across datasets: every BR row dominates every baseline row on every metric. BR-0 alone reduces imbalance by $4.1\times$ on Proprietary Data and $1.94\times$ on Azure-2024 over the strongest baseline (JSQ), lifting throughput by 11.8% and 8.8% respectively. Oracle BR-H tightens imbalance by a further $\sim 2\times$ on Proprietary Data and $\sim 1.5\times$ on Azure, reaching $\sim 1,029$ and 957 tok/s; the gap between BR-0 and oracle BR-H quantifies the structural reward available to lookahead under perfect prediction (Section 4.1). The deployed

ExactMatch predictor recovers most of this lookahead gain end-to-end (977 and 909 tok/s, +15.4% and +14.5% over the strongest baseline), and Survival delivers comparable throughput on both traces. ExactMatch is the primary deployment for workloads with prompt-level recurrence (Appendix C.3 characterizes it as aligned with this regime) while Survival is the principled fallback when recurrence is absent. Deployed BR-H also achieves the lowest TPOT P95 of any method on both datasets, confirming that throughput gains do not come at the cost of tail latency. Per-worker KV-workload visualizations and the trace-level Survival analysis are in Appendices F and E.

6.3 Scaling with system size

We measure how each router’s performance evolves with G on three configurations: $G=4$ on 2P1D (48 NPUs), $G=8$ on 4P1D (96 NPUs, the main configuration), and $G=16$ on 5P1D (144 NPUs). Per-worker offered load is held constant by scaling request rate proportionally with G . To isolate router behaviour from predictor-quality effects, we run BR-H with oracle prediction at the operating point $(\beta, \gamma)=(14.67, 0.64)$ used in the $G=8$ main results.

Table 2: **Scaling summary on Proprietary Data.** $G=4$ (2P1D, 48 NPUs), $G=8$ (4P1D, 96 NPUs), $G=16$ (5P1D, 144 NPUs). BR-H runs with oracle prediction at $H=80$, $(\beta, \gamma)=(14.67, 0.64)$. Each cell reports avg. imbalance / TPOT P95 / throughput. Best in each column in **bold**.

Method	$G=4$ (2P1D)	$G=8$ (4P1D)	$G=16$ (5P1D)
Random	80,118 / 66.30 / 364.7	438,225 / 85.15 / 810.5	638,062 / 77.80 / 923.6
Round-Robin	68,108 / 65.27 / 365.7	254,798 / 83.16 / 846.7	616,089 / 77.91 / 925.3
P2C	64,012 / 65.40 / 366.4	262,940 / 84.01 / 836.3	688,283 / 77.46 / 927.3
JSQ	75,326 / 66.45 / 365.4	215,110 / 83.20 / 842.8	675,607 / 77.40 / 926.6
BR-0	9,711 / 63.19 / 407.1	51,927 / 79.32 / 942.5	215,170 / 74.92 / 1,117.8
BR-H <i>oracle</i>	6,576 / 62.52 / 415.3	23,577 / 78.04 / 1,028.9	117,067 / 72.04 / 1,247.4

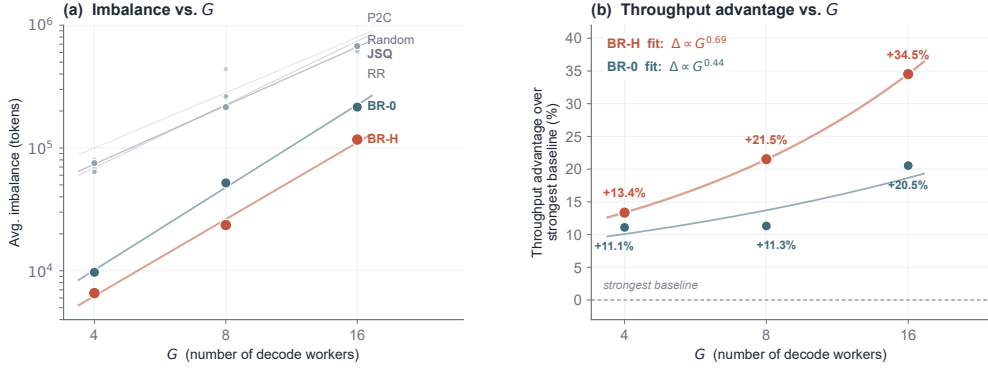


Figure 4: **Scaling on Proprietary Data across $G \in \{4, 8, 16\}$.**

Table 2 and Figure 4 report the headline numbers. BR-H stays well below every baseline at every tested G : 6.6k tokens at $G=4$ vs. 64–80k for baselines, 24k at $G=8$ vs. 215–438k, and 117k at $G=16$ vs. 616–688k. All bands grow with G , consistent with the order-statistics argument of Section 2.1. The throughput advantage over the strongest baseline rises with G : +13.4% at $G=4$, +21.5% at $G=8$, +34.5% at $G=16$. BR-0’s advantage follows a similar but flatter trajectory (+11%, +11%, +20.5%); the BR-H-over-BR-0 gap also widens with G (+2%, +9%, +12%). Absolute throughput at $G=16$ is sublinear because 5P1D adds only one prefill node for twice the decode workers, dropping the prefill-to-decode ratio from 4:1 to 5:2; per-worker advantages at fixed G are unaffected. Per-worker traces underlying every cell are in Appendix G.

6.4 Robustness and dispatch overhead

Sensitivity to (β, γ) . BR-H is robust across the explored parameter region. We sweep a cross-shape around $(\beta=48, \gamma=0.9)$ on Proprietary Data with $\beta \in \{1, 24, 48, 96\}$ at $\gamma=0.9$ and $\gamma \in$

$\{0.5, 0.7, 0.9, 1.0\}$ at $\beta=48$ (seven configurations, oracle, $H=80$). Throughput across the sweep ranges 945–1,042 tok/s and TPOT P95 ranges 77.3–79.5 ms; every swept configuration improves on the strongest baseline by a wide margin. Full sweep tables (Table 6,7), Figure 13 and per-worker traces are in Appendix H.

Dispatch overhead. Figure 5 in Appendix D.7 measures the per-tick cost of the BR-H dispatcher itself. We instrument the proxy on two deployments—DeepSeek-V3 671B and Qwen3-30B-A3B, both at $G=8$, $R_{\max}=4$. Total per-loop dispatch cost is $P_{50}=1.20$ ms (mean 1.34 ms, $P_{99}=2.77$ ms) on DeepSeek-V3 and $P_{50}=1.03$ ms ($P_{99}=2.52$ ms) on Qwen3, the two distributions agreeing within 17% at P_{50} . Against the ~ 60 ms per-step engine budget, this places the dispatcher $\sim 50\times$ below the engine step at the median and $\sim 22\times$ below at P_{99} . The per-stage decomposition (Stage 2 dominates, $\sim 69\%$ of total) is in Appendix D.

7 Conclusion, Limitations, and Future Work

We presented **BalanceRoute**, a family of practical online routers for DP-decode load balancing. Evaluated on a 144-NPU cluster against four standard load-balancing baselines on both a proprietary production trace and the public Azure-2024 trace, BalanceRoute substantially reduces DP imbalance and improves end-to-end throughput, with the advantage over the strongest baseline rising from +13.4% at $G=4$ to +34.5% at $G=16$.

Limitations and future works. Each cell is a single replay; consistent 144-NPU availability precludes systematic replicate sweeps, though qualitative orderings are stable across two datasets and three cluster sizes. A potential future direction is to overlap cross-tier KV transfer with decode under $H>0$ to characterise how predictor noise interacts with G across the full range of cluster sizes, and to explore decentralised or hardware-offloaded variants of the control plane.

References

- Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. *arXiv preprint arXiv:2403.02310*, 2024.
- Ruicheng Ao, Gan Luo, David Simchi-Levi, and Xinshang Wang. Optimizing llm inference: Fluid-guided online scheduling with memory constraints. *arXiv preprint arXiv:2504.11320*, 2025.
- Zixi Chen, Yinyu Ye, and Zijie Zhou. Adaptively robust llm inference optimization under prediction uncertainty. *arXiv preprint arXiv:2508.14544*, 2025.
- Zixi Chen, Tianci Bu, Chendong Song, Xin Lu, Yinyu Ye, and Zijie Zhou. A universal load balancing principle and its application to large language model serving. *arXiv preprint arXiv:2601.17855*, 2026.
- DeepSeek. Expert parallelism load balancer (eplb). GitHub repository, 2025a. URL <https://github.com/deepseek-ai/EPLB>. Expert Parallelism Load Balancer.
- DeepSeek. Lplb: Linear-programming-based load balancer for mixture-of-experts models. GitHub repository, 2025b. URL <https://github.com/deepseek-ai/LPLB>. An early research stage expert-parallel load balancer for MoE models based on linear programming.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient llm scheduling by learning to rank. *arXiv preprint arXiv:2408.15792*, 2024.
- Patrick Jaillet, Jiashuo Jiang, Konstantina Mellou, Marco Molinaro, Chara Podimata, and Zijie Zhou. Online scheduling for llm inference with kv cache constraints. *arXiv preprint arXiv:2502.07115*, 2025.
- Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, et al. Intelligent router for llm workloads: Improving performance through workload-aware scheduling. *arXiv preprint arXiv:2408.13510*, 2024.
- Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, et al. Performance aware llm load balancer for mixed workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, pages 19–30, 2025.
- David R Karger, Steven J Phillips, and Eric Torng. A better algorithm for an ancient scheduling problem. *Journal of Algorithms*, 20(2):400–430, 1996.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Microsoft Azure Project. Microsoft azure traces. <https://github.com/Azure/AzurePublicDataset>, 2025. Open-source Microsoft Azure traces.
- Michael Mitzenmacher. The power of two choices in randomized load balancing. *IEEE transactions on parallel and distributed systems*, 12(10):1094–1104, 2002.
- Miray Özcan, Philipp Wiesner, Philipp Weiß, and Odej Kao. Quantifying the energy consumption and carbon emissions of llm inference via simulations. *arXiv preprint arXiv:2507.11417*, 2025.

- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. *Power*, 400(700W): 1–75, 2023.
- Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX conference on file and storage technologies (FAST 25)*, pages 155–170, 2025.
- Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew T Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Efficient interactive llm serving with proxy model-based sequence length prediction. *arXiv preprint arXiv:2404.08509*, 2024.
- Rana Shahout, Eran Malach, Chunwei Liu, Weifan Jiang, Minlan Yu, and Michael Mitzenmacher. Don’t stop me now: Embedding based scheduling for llms. *arXiv preprint arXiv:2410.01035*, 2024.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing llm inference clusters for performance and energy efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pages 1348–1362. IEEE, 2025.
- vLLM Community. *vLLM Ascend Plugin*. vLLM Project, 2025. URL <https://docs.vllm.ai/projects/ascend/en/latest/>.
- vLLM Contributors. Data parallel deployment. https://docs.vllm.ai/en/latest/serving/data_parallel_deployment/, May 2026.
- vLLM Project. vllm router. <https://github.com/vllm-project/router>, 2025. Open-source request router for vLLM serving stack.
- Meixuan Wang, Yinyu Ye, and Zijie Zhou. Llm serving optimization with variable prefill and decode lengths. *arXiv preprint arXiv:2508.06133*, 2025.
- Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37: 62557–62583, 2024.
- Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response length perception and sequence scheduling: An llm-empowered llm inference pipeline. *Advances in Neural Information Processing Systems*, 36:65517–65530, 2023.
- Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. *arXiv preprint arXiv:2401.09670*, 2024.
- Zijie Zhou. Position: Llm serving needs mathematical optimization and algorithmic foundations, not just heuristics, 2026. URL <https://arxiv.org/abs/2605.01280>.

A Other Related Works

This appendix expands on related work that lies outside the immediate scope of our routing problem but situates our contribution in the broader LLM-serving literature.

A.1 LLM serving systems and architecture

Recent serving systems have rearchitected the inference stack to expose structural levers for throughput and latency. vLLM introduces PagedAttention to virtualize and compact the KV cache [Kwon et al., 2023], eliminating fragmentation and enabling flexible large batches with near-zero KV waste; SGLang co-designs a program/runtime interface with batching and caching [Zheng et al., 2024] for multi-call LLM applications. At the cluster level, prefill-decode (PD) disaggregation separates prompt encoding from autoregressive decoding across worker pools [Zhong et al., 2024, Patel et al., 2023] so that TTFT and TPOT can be optimized independently. These structural choices interact with parallelism: decode commonly uses data parallelism (DP) over requests, while model parallelism mixes tensor parallelism (TP) [Shoeybi et al., 2019] for dense layers, expert parallelism (EP) [Fedus et al., 2022, Liu et al., 2024] for MoE layers, and pipeline parallelism for very large models. Together, these form the substrate on which scheduling and load-balancing policies operate; our work targets the policy layer that sits on top of this substrate.

A.2 Single-worker scheduling and batching

Production engines adopt iteration-level scheduling with continuous batching as the default control primitive. Orca introduces iteration-level scheduling and selective batching to keep accelerators busy under heterogeneous request lengths [Yu et al., 2022], and vLLM co-designs PagedAttention with preemptive request scheduling at iteration granularity [Kwon et al., 2023]. Building on these primitives, several systems explore complementary schedulers: Sarathi-Serve mitigates prefill-decode interference via chunked prefill [Agrawal et al., 2024]; FastServe adds token-granular preemption via a multi-level feedback queue to reduce job completion time [Wu et al., 2023]. A parallel line uses learned or proxy signals to approximate shortest-job-first scheduling, including proxy-model predictors [Qiu et al., 2024] and embedding-based estimators [Shahout et al., 2024]. These works optimize *within* a single worker—how to batch and order requests already routed to it—and are complementary to our cross-worker routing problem.

A.3 Cross-worker routing and load balancing

A growing body of work targets routing requests across multiple LLM workers, with several lines of attack. *Generic load-balancing heuristics* dominate production deployments: the vLLM router [vLLM Project, 2025] and SGLang router [Zheng et al., 2024] implement round-robin, random, power-of-two-choices, consistent hashing, and prefix-cache-aware routing—policies that originate from classical distributed-systems load balancing [Mitzenmacher, 2002, Karger et al., 1996] and are agnostic to the LLM-specific structure that our paper exploits. *Workload-aware request routing* systems treat LLM instances as independent endpoints and optimize end-to-end latency by routing each arriving request to the most suitable instance. The Intelligent Router of Jain et al. [2024] and the follow-up Performance-Aware LLM Load Balancer [Jain et al., 2025] exemplify this thread: both train an output-length classifier (DistillBERT) to bucket request decode lengths, derive an analytical workload-mixing penalty that estimates within-instance prefill-decode interference, and use a heuristic-guided reinforcement-learning agent to choose the target instance. They report up to 11% end-to-end latency reduction over round-robin on small clusters (4–8 LLM instances). *MoE expert balancing* addresses imbalance *within* a worker that uses expert parallelism, redistributing tokens across redundant experts [DeepSeek, 2025a,b] or via dynamic prefetching; these operate at a different granularity from cross-replica DP balancing.

The DP-decode setting we address differs from each of these threads along an axis that none of them captures: *barrier synchronization*. Workload-aware request routers [Jain et al., 2024, 2025] treat each LLM instance as an independent endpoint whose latency depends only on its own active batch; the optimization target is end-to-end response latency, and a slow instance harms only the requests it serves. In our setting, each DP replica is parametric-sharded with TP/EP and tied to all other replicas through a step-wise collective barrier, so a slow replica delays *every* replica at

every decode step. The routing decision is a fleet-level imbalance-control problem rather than an instance-selection problem, and the relevant time scale is the sub-100 ms decode step rather than the request lifetime. RL-based routers with non-trivial inference overhead per decision are not operationally viable at this granularity, and instance-selection objectives such as “best target for this request” do not capture imbalance-induced barrier idle. Chen et al. [2026] are the first to formalize the DP-decode load-balancing problem with explicit barrier synchronization, and Zhou [2026] highlight it as a representative LLM-serving problem requiring principled algorithm design beyond generic heuristics. Our work develops practical schedulers in this setting with deployable per-step cost.

A.4 Theoretical foundations for LLM scheduling

A recent theory stream formalizes LLM serving as online scheduling with KV-cache constraints. Jaillet et al. [2025] develop a hindsight-optimal integer-program benchmark, prove impossibility in the fully online adversarial setting, and design near-optimal algorithms under a semi-online model; Chen et al. [2025] and Wang et al. [2025] relax assumptions in this model, addressing heterogeneous prefill, unknown decode lengths with interval predictions, and variable prefill–decode lengths. Ao et al. [2025] introduce a fluid-guided framework with memory constraints and derive Nested-WAIT policies that are near-optimal against a fluid benchmark in heavy traffic. These works provide principled scheduling *within a single worker*; the multi-worker setting under barrier synchronization—where our paper operates—is the focus of Chen et al. [2026] and the present work.

B Algorithm Pseudocode

In this section, we provide algorithm pseudocode for BR-0 and BR-H respectively:

Algorithm 2: BalanceRoute-0 (BR-0)

Input: Waiting set $R_{\text{wait}}(k)$, active sets $\{\mathcal{A}_g(k)\}$, batch limit B , thresholds $S_{\text{greedy}}, R_{\text{max}}$

Output: Assignments $\{S_g(k)\}_{g \in [G]}$ at step k

$\text{cap}[g] \leftarrow B - |\mathcal{A}_g(k)|$ for all g ; $S_{\text{tot}} \leftarrow \sum_g \text{cap}[g]$;

if $S_{\text{tot}} = 0$ **or** $R_{\text{wait}}(k) = \emptyset$ **then return**;

$M(k) \leftarrow \max_g L_g(k)$; $m_g \leftarrow M(k) - L_g(k)$ for all g ;

Stage 1: Greedy fill;

while $S_{\text{tot}} > S_{\text{greedy}}$ **and** $R_{\text{wait}}(k) \neq \emptyset$ **do**

$g \leftarrow \arg \max_{g'} \text{cap}[g']$; // tie-break: smallest $L_{g'}(k)$
 $i^* \leftarrow \arg \max_{i \in R_{\text{wait}}(k)} F_g(\{i\})$;
 $\mathcal{A}_g(k) \leftarrow \mathcal{A}_g(k) \cup \{i^*\}$; $R_{\text{wait}}(k) \leftarrow R_{\text{wait}}(k) \setminus \{i^*\}$;
 update $\text{cap}[g], S_{\text{tot}}, L_g(k), M(k), \{m_{g'}\}$;

Stage 2: Refined allocation;

Initialize priority queue $\mathcal{Q}$ over $\{g : \text{cap}[g] > 0\}$, keyed by $(\text{cap}[g], m_g)$;

while $\mathcal{Q} \neq \emptyset$ **and** $R_{\text{wait}}(k) \neq \emptyset$ **do**

Pop g from $\mathcal{Q}$; $c_g \leftarrow \text{cap}[g]$;
 Take up to R_{max} candidates from head of $R_{\text{wait}}(k)$ (sorted by s_i);
 $Q^* \leftarrow \arg \max_{|Q| \leq \min(c_g, R_{\text{max}})} F_g(Q)$ over candidate subsets;
if $F_g(Q^*) \leq 0$ **then** $Q^* \leftarrow \{\arg \max_i F_g(\{i\})\}$; // starvation guard
 ;
 $\mathcal{A}_g(k) \leftarrow \mathcal{A}_g(k) \cup Q^*$; $R_{\text{wait}}(k) \leftarrow R_{\text{wait}}(k) \setminus Q^*$;
 update $\text{cap}[g], L_g(k), M(k), \{m_{g'}\}$; **if** $\text{cap}[g] > 0$ **then** reinsert g into $\mathcal{Q}$;

Algorithm 3: BalanceRoute-H (BR-H)

Input: Waiting set $R_{\text{wait}}(k)$, active sets $\{\mathcal{A}_g(k)\}$, batch limit B , horizon H , prediction period ΔT (default as $H/2$) thresholds $S_{\text{greedy}}, R_{\text{max}}$, score parameters (α, β, γ)

Output: Assignments $\{S_g(k)\}_{g \in [G]}$ at step k

$\text{cap}[g] \leftarrow B - |\mathcal{A}_g(k)|$ for all g ; $S_{\text{tot}} \leftarrow \sum_g \text{cap}[g]$;

if $S_{\text{tot}} = 0$ **or** $R_{\text{wait}}(k) = \emptyset$ **then return**;

For each $i \in \bigcup_g \mathcal{A}_g(k)$, update the cached $\hat{c}_i$ every ΔT decode steps: recompute via (6); derive

$\{L_g(k+h)\}_{h=0}^H$, envelope $\mathbf{M}$, margins $\{\mathbf{m}_g\}$;

Stage 1: Greedy fill;

while $S_{\text{tot}} > S_{\text{greedy}}$ **and** $R_{\text{wait}}(k) \neq \emptyset$ **do**

$g \leftarrow \arg \max_{g'} \text{cap}[g']$; // tie-break: smallest $L_{g'}(k)$

$i^* \leftarrow \arg \max_{i \in R_{\text{wait}}(k)} F_g(\{i\})$;

$\mathcal{A}_g(k) \leftarrow \mathcal{A}_g(k) \cup \{i^*\}$; $R_{\text{wait}}(k) \leftarrow R_{\text{wait}}(k) \setminus \{i^*\}$;

 update $\text{cap}[g]$, S_{tot} , projected loads, $\{\mathbf{m}_{g'}\}$;

Stage 2: Refined allocation;

Initialize priority queue $\mathcal{Q}$ over $\{g : \text{cap}[g] > 0\}$, keyed by $(\text{cap}[g], \min_h m_{g,h})$;

while $\mathcal{Q} \neq \emptyset$ **and** $R_{\text{wait}}(k) \neq \emptyset$ **do**

 Pop g from $\mathcal{Q}$; $c_g \leftarrow \text{cap}[g]$;

 Take up to R_{max} candidates from head of $R_{\text{wait}}(k)$;

$Q^* \leftarrow \arg \max_{|Q| \leq \min(c_g, R_{\text{max}})} F_g(Q)$ over candidate subsets;

if $F_g(Q^*) \leq 0$ **then** $Q^* \leftarrow \{\arg \max_i F_g(\{i\})\}$; // starvation guard

 ;

$\mathcal{A}_g(k) \leftarrow \mathcal{A}_g(k) \cup Q^*$; $R_{\text{wait}}(k) \leftarrow R_{\text{wait}}(k) \setminus Q^*$;

 update $\text{cap}[g]$, projected loads, $\{\mathbf{m}_{g'}\}$; **if** $\text{cap}[g] > 0$ **then** reinsert g into $\mathcal{Q}$;

C Prediction interface

C.1 Short-horizon prediction interface

Fix a horizon H . For each active request $i \in \bigcup_g \mathcal{A}_g(k)$ at step k , BR-H requires a single bounded scalar

$$\hat{c}_i(k) \approx \mathbb{E}[\min(r_i(k), H) \mid x_i, a_i(k)] \in [0, H], \quad (4)$$

the *expected in-window contribution* of i to the next H decode steps. The choice of estimand is dictated by what BR-H actually consumes: the horizon F-score (Section 4.1) reads only how much in-window work each request contributes, truncated at H both because the discount γ^h already attenuates beyond it and because longer horizons are dominated by unobservable future arrivals. Anything finer than $\hat{c}_i(k)$ is not used; anything coarser loses signal at the binary threshold “ $r_i(k) \leq H$ ”.

The estimand admits a clean two-stage decomposition. Conditioning on the event $A_i := \{r_i(k) \leq H\}$ that request i finishes inside the window, and using $\min(r_i(k), H) = r_i(k)$ on A_i and H on A_i^c ,

$$\mathbb{E}[\min(r_i(k), H) \mid x_i, a_i(k)] = (1 - p_{\text{fin}}^{(i)}) \cdot H + p_{\text{fin}}^{(i)} \cdot \mu_{\text{rem}}^{(i)}, \quad (5)$$

where $p_{\text{fin}}^{(i)} := \Pr(r_i(k) \leq H \mid x_i, a_i(k))$ and $\mu_{\text{rem}}^{(i)} := \mathbb{E}[r_i(k) \mid x_i, a_i(k), r_i(k) \leq H] \in (0, H]$. The decomposition gives BR-H two estimators with naturally aligned losses:

1. *Termination classifier.* A calibrated estimate $\hat{p}_{\text{fin}}^{(i)}$ of $p_{\text{fin}}^{(i)}$, trained with cross-entropy. Calibration matters because the composite reads $\hat{p}_{\text{fin}}^{(i)}$ as a probability rather than a hard label.
2. *Conditional-mean regressor.* An estimate $\hat{\mu}_{\text{rem}}^{(i)}$ of $\mu_{\text{rem}}^{(i)}$, trained with squared error on the finish-positive subsample. Squared error is the proper loss because the target is a conditional mean.

The composite estimate is

$$\hat{c}_i(k) = (1 - \hat{p}_{\text{fin}}^{(i)}) \cdot H + \hat{p}_{\text{fin}}^{(i)} \cdot \hat{\mu}_{\text{rem}}^{(i)}, \quad (6)$$

clipped to $[0, H]$. When $\hat{p}_{\text{fin}}^{(i)}$ is near zero the composite collapses to the conservative anchor H ; when near one it tracks $\hat{\mu}_{\text{rem}}^{(i)}$.

The interface is deliberately agnostic to how $\hat{p}_{\text{fin}}^{(i)}$ and $\hat{\mu}_{\text{rem}}^{(i)}$ are realized. Anything implementing the two-stage contract above plugs in: simple non-parametric estimators based on the empirical training-output CDF, per-prompt memorization, or learned classifier-and-regressor pairs are all valid instantiations, and the BR-H rule does not distinguish among them. We describe specific estimator families used in our deployment, together with their accuracy characterizations, in Section 6; full implementation details are deferred to Appendix C.2.

From $\hat{c}_i(k)$ we obtain the projected per-step workload that the score consumes by aging the contribution down to zero across the window:

$$\hat{w}_i^{(h)}(k) \approx \begin{cases} s_i + a_i(k) + h - 1, & h \leq \hat{c}_i(k), \\ 0, & h > \hat{c}_i(k), \end{cases} \quad h = 1, \dots, H, \quad (7)$$

which preserves the convention that an active request contributes $s_i + a_i(k) + h - 1$ at step $k + h$ and contributes nothing once it has departed within the window. Predictions are refreshed every ΔT decode steps and decremented by 1 per step in between; we defer the choice of ΔT and the remaining details (confidence gating, floor handling) to Appendix C.2.

Aggregating per-request profiles over each worker’s active set yields a horizon-projected load trajectory $\{L_g(k + h)\}_{h=0}^H$, an envelope $M_h := \max_{g'} L_{g'}(k + h)$, and a margin vector

$$\mathbf{m}_g := ((M_h - L_g(k + h))_+)_{h=0}^H \in \mathbb{R}_{\geq 0}^{H+1}.$$

All three are computed once at the start of each scheduling round and updated incrementally as admissions are made.

C.2 Implementation of the two-stage prediction interface

The two-stage interface specified in Section C.1 is a contract between Stage 1 and Stage 2: a calibrated probability $\hat{p}_{\text{fin}}^{(i)}$ paired with a conditional-mean estimate $\hat{\mu}_{\text{rem}}^{(i)} \in (0, H]$, combined into the composite $\hat{c}_i(k)$ via (6). The contract is agnostic to how the two estimates are realized; multiple concrete realizations satisfy it, and the BR-H rule does not distinguish among them. This appendix describes the realizations evaluated in Section 6, the training protocol, the refresh logic that maintains $\hat{c}_i(k)$ as a request progresses, and the latency budget at deployment.

C.2.1 Estimator realizations

We evaluate three realizations spanning the spectrum of training cost and feature engineering. All three implement the same Stage-1 / Stage-2 contract.

Empirical-survival. The empirical-survival estimator reads both stages directly off the empirical training-output CDF $\hat{F}$. Stage 1 is

$$\hat{p}_{\text{fin}}^{(i)} = \frac{\hat{F}(a_i(k) + H) - \hat{F}(a_i(k))}{1 - \hat{F}(a_i(k))},$$

the conditional probability that an output sampled from the training distribution would terminate within the next H steps given that it has already produced $a_i(k)$ tokens. Stage 2 is the conditional mean

$$\hat{\mu}_{\text{rem}}^{(i)} = \frac{1}{|\{j : a_i(k) < o_j \leq a_i(k) + H\}|} \sum_{a_i(k) < o_j \leq a_i(k) + H} (o_j - a_i(k))$$

on the same training corpus. Both stages have $O(\log n)$ inference cost on a sorted training output history and require no per-request features. This is the production default when no auxiliary signal is available.

Per-prompt-class memorization. When the workload exhibits prompt repetition (multi-turn dialogue, cached prompts, structured templates), a per-prompt empirical CDF tightens both stages substantially over the marginal CDF. We maintain a key-indexed CDF—key derived from a hash of the prompt, or a quantile bucket of s_i —and apply the empirical-survival formulas above within the matching bucket; on key miss, the estimator falls back to the marginal empirical-survival baseline. Inference cost is $O(\log n)$ per call with no feature pipeline.

Learned classifier and regressor. For workloads with content-aware signal beyond the marginal output distribution, we train a learned classifier-and-regressor pair. Stage 1 is a binary classifier minimizing cross-entropy on the label $\mathbb{K}[r_i(k) \leq H]$. Stage 2 is a regressor minimizing squared error on the finish-positive subsample with target $r_i(k) \in (0, H]$. Both estimators consume a feature vector built from $s_i, a_i(k)$, exponentially-weighted moving averages of recent observed output lengths, and rolling-window statistics on past outputs and inputs; features are causal by construction. Concrete model classes are pluggable; in Section 6 we use gradient-boosted trees with 200 estimators of depth 3, but the interface admits MLP or other replacements without affecting the rest of the pipeline.

C.2.2 Training protocol

For the empirical-survival and memorization variants, “training” reduces to sorting an output-length history and is $O(n \log n)$. For the learned variant, training samples are synthesized by walking each historical (s_j, o_j) at age points $T = 0, \Delta T, 2\Delta T, \dots$ up to $T < o_j$, emitting one (features, $\mathbb{K}[o_j \leq T + H], o_j - T$) tuple per age point; the classifier trains on all tuples and the regressor on the finish-positive subset. We use $\Delta T = H/2$, which controls training-set density without inflating cost.

For each evaluated dataset we hold out a time-disjoint segment for training and a separate segment for evaluation. Stage-1 metrics (ROC-AUC, Brier score) are computed on the binary task; Stage-2 metrics (conditional MAE) are computed on the finish-positive subsample. Distribution drift is handled by periodic re-fit; the cost of re-fit is dominated by the sort for non-parametric variants and by gradient boosting for the learned variant.

C.2.3 Refresh logic

The proxy maintains $\hat{c}_i(k)$ along three rules.

Periodic refresh. The proxy re-queries the predictor every ΔT generated tokens (default $\Delta T = H/2$), recomputing $\hat{p}_{\text{fin}}^{(i)}$ and $\hat{\mu}_{\text{rem}}^{(i)}$ at the current age and updating $\hat{c}_i(k)$ via (6). Between refreshes the proxy decrements $\hat{c}_i(k)$ by one per generated token, since each elapsed step is one less step of in-window contribution.

Stage-1 confidence gate. On each refresh, the predictor’s update is accepted only when $\hat{p}_{\text{fin}}^{(i)} \geq 0.5$; otherwise $\hat{c}_i(k)$ is reset to the conservative anchor H . The gate is necessary because the regressor’s output is well-defined only on the finish-positive subsample; on long-tail requests where $p_{\text{fin}}^{(i)}$ is small, $\hat{\mu}_{\text{rem}}^{(i)}$ is effectively unconstrained and a refresh that uses it would inject a phantom near-departure event into the projection. Gating on $\hat{p}_{\text{fin}}^{(i)}$ rather than on the magnitude of $\hat{c}_i(k)$ avoids this failure mode at the natural decision boundary of the two-stage decomposition.

Floor. While the request remains active, $\hat{c}_i(k)$ is bounded below by 1. A floor of zero would let an over-confident initial prediction silently mark the request as already departed between scheduled refreshes, dropping it from the projection while it is still consuming work; the floor closes this gap. Crossing the floor between scheduled refreshes triggers an immediate refresh, so a request that was incorrectly predicted to finish soon is corrected at the moment its prediction would expire.

C.2.4 Latency budget

Per-call inference cost is $\sim 1 \mu\text{s}$ for the empirical-survival and memorization variants (a binary search on a sorted array) and $\sim 1 \text{ ms}$ for the learned variant (a gradient-boosted tree forward pass; feature extraction dominates). For the learned variant, refresh latency is amortized by batching all active-request refreshes that fall on a common ΔT boundary into a single forward pass; under the deployed dispatcher this places predictor cost an order of magnitude below the per-tick scheduling budget of Section D.4.

C.3 Predictor evaluation

The two-stage interface of Section C.1 requires a calibrated probability $\hat{p}_{\text{fin}}^{(i)}$ of in-window finish and a conditional-mean estimate $\hat{\mu}_{\text{rem}}^{(i)} \in (0, H]$ on the finish-positive subsample, combined into $\hat{c}_i(k)$ via (6). We deploy two concrete realizations of this interface that share the same admission features, refresh contract, and composite formula, and differ only in how each stage is fit. **Empirical-survival** reads both stages off the marginal training-output CDF in $O(\log n)$ per call with no per-request features. **Per-prompt memorization (ExactMatch)** maintains a prompt-hash-keyed empirical CDF and applies the same formulas within the matching bucket, falling back to the marginal CDF on key miss. ExactMatch is therefore a generalization of Empirical-survival: equivalent to it on every request whose prompt has not been seen in training and tighter only when prompt-level recurrence is present. Both realizations share the same proxy binary and are toggled by a configuration flag.

Table 3 reports the offline accuracy of both realizations on both workloads. The two metrics read together: Stage-1 AUC is inflated by the positive-class base rate when most active requests sit within H , while Stage-2 conditional MAE captures regressor quality on the finish-positive subsample. Outputs concentrating near a cap (Azure) make both stages tight regardless of realization. On Proprietary Data, Empirical-survival sits at the floor reachable from the marginal output distribution alone, while ExactMatch tightens both stages by exploiting per-prompt structure where it exists. The two rows on Proprietary Data bracket what is achievable: Empirical-survival is the lower bound that does not depend on prompt structure; ExactMatch exploits this information effectively.

Both realizations have sub-10ms per-call inference cost. Refreshes occur at the $\Delta T = H/2$ boundary on the SSE-parsing path inside the proxy (Section D.3), so predictor work amortizes over chunks the proxy is already inspecting and stays well below the per-tick scheduling budget of Section D.4.

Table 3: **Offline predictor accuracy.** Stage 1: ROC-AUC on $\{r_i(k) \leq H\}$. Stage 2: conditional MAE in tokens on the finish-positive subsample.

Dataset	Realization	Stage-1 AUC $\uparrow$	Stage-2 MAE $\downarrow$
Azure-2024	Empirical-survival	0.993	5.4
Azure-2024	Per-prompt memorization	0.995	5.4
Proprietary Data	Empirical-survival	0.700	20.1
Proprietary Data	Per-prompt memorization	0.974	2.9

D Engineering details of the system implementation

This appendix supplies the implementation details deferred from Section 5. Section D.1 expands the proxy’s data structures and the coroutines that maintain them. Section D.2 gives the precise payload-mutation contract used to drive the prefill–decode hand-off, including retry, cancellation, and recomputation handling. Section D.3 explains the in-band telemetry. Section D.4 describes the bit-set construction behind Stage-2 subset selection and analyses its complexity. Section D.5 describes the optional engine-side instrumentation we use for closed-loop validation. Section D.6 describes the pool-bypass deployment path for BR-0. Section D.7 displays the dispatch overhead in our experiment.

D.1 Proxy data structures and coroutines

The structures introduced in Section 5 have the following implementation notes.

DecodeInstanceState[g]. A dictionary of RequestTracker entries keyed by request id. Each tracker also stores the producer-side hand-off metadata returned by the prefill probe (Section D.2), which is needed to re-attach the request at decode admission. The aggregate $\text{cap}[g] = B - |\mathcal{A}_g(k)|$ is maintained alongside $L_g(k)$ for $O(1)$ lookup at dispatch time.

PromptPool. An arrival-ordered registry of prefill-completed entries awaiting decode admission. Each entry holds a tracker plus the producer-side hand-off metadata, so admission to a decode worker

requires no further round-trip to the prefill tier. Entries also carry a TTL that bounds the time a request may sit in the pool waiting for a slot; expired entries are aborted by a pool-reaper coroutine, which also issues a release on the corresponding prefiller-side delayed-free record.

`PrefillerBudget[i]`. The most recent KV-cache utilization sample of prefiller i . A parse failure or scrape timeout is treated conservatively as utilization 1.0, so a transiently unreachable prefiller is automatically excluded from rotation rather than mistakenly preferred for its stale low reading.

Cooperating coroutines. Four coroutines drive the pipeline. (i) A *prefill dispatcher* dequeues client arrivals, picks an unsaturated prefiller, and forks an `asyncio` task to issue the prefill probe and insert the completed entry into the pool. (ii) A *decode dispatcher* blocks on a pair of `asyncio.Events` (`pool_not_empty` and `decoder_slot_freed`) and, on wake, runs Algorithm 3 against the current pool and decode-worker grid, emitting a batch of admissions in one tick. (iii) A *metrics poller* reads each prefiller’s Prometheus endpoint at a fixed cadence and updates `PrefillerBudget`. (iv) A *pool reaper* ages out entries past their TTL.

Connection pooling. Each `vllm-ascend` worker is fronted by a dedicated `httpx.AsyncClient` with a generously sized keep-alive pool over persistent HTTP/1.1 connections, so TCP setup is amortized across many requests rather than paid per dispatch. SSE streams from decoders are forwarded chunk-by-chunk to clients while being parsed inline for state mirroring.

D.2 Payload-mutation protocol

Prefill probe. The proxy clones the incoming client request and applies three overrides:

- `stream=False`,
- `max_tokens=min_tokens=1`,
- `kv_transfer_params={do_remote_decode: true, do_remote_prefill: false, ...}`.

The first two overrides force the prefiller to compute the full KV cache, return one token (so the response carries usage statistics, including the realized prompt length), and stop short of autoregressive decode. The third instructs `MooncakeConnectorV1` to retain the KV blocks on the producer until the consumer signals receipt, and to populate the response with the producer-side handles `remote_engine_id`, `remote_block_ids`, and `remote_host:remote_port`. The proxy reads the realized `prompt_tokens` from the response `usage` field and records it on the request’s tracker, then parks the entry in the pool together with the returned `kv_transfer_params`.

Decode admission. When BR-H selects an entry for decode worker g , the proxy reconstitutes the original request—`stream=True`, original `max_tokens`, original sampling parameters—and re-attaches the producer-side `kv_transfer_params`. Worker g enters `WAITING_FOR_REMOTE_KVS`; `MooncakeConnectorV1` pulls the KV blocks from the producer and, on completion, emits a `DONE_RECVING_MSG` that releases the prefiller’s delayed-free queue. The first decoded chunk arriving at the proxy is the unambiguous signal that the cross-tier transfer has completed; the proxy uses it to release the prefiller’s KV-cache budget for the next request.

Recomputation handling. A by-product of the payload-mutation protocol is a clean handler for `vLLM`’s `stop_reason=recomputed` event, which can occur under preemption: the proxy detects it inline, unregisters the request from worker g , augments the original prompt with the already-emitted tokens, decrements `max_tokens` accordingly, and re-enters the request as a fresh pool entry, preserving end-to-end completion semantics without engine modification.

Cancellation. A `listen_for_disconnect` side-task ensures that a client cancellation—whether while the entry is in the pool or after dispatch but before the first decoded token—deterministically releases the corresponding pool slot or decode-worker slot rather than leaking a sticky assignment. Retries follow exponential backoff with capped attempts.

D.3 In-band telemetry

BR-H requires two streams of feedback that are not exposed by the standard OpenAI-compatible API.

Per-token decode progress. As the proxy forwards SSE chunks from the decoder to the client, it parses each one inline. A content delta triggers a simple increment of the $a_i(k)$ counter for that request; a finish signal triggers cleanup. The heavier predictor recomputation (Section C.1) is gated to ΔT -step boundaries rather than being run per token, so the per-chunk work remains an integer update plus a marker check; parsing happens before the chunk leaves the proxy, so no token-level latency is added on the critical path.

Prefiller KV budget. The delayed-free mechanism in MooncakeConnectorV1 holds KV caches on the prefiler until a consumer commits to downloading them. To prevent a prefiler from blocking on full cache, a metrics poller reads the standard Prometheus endpoint of each prefiler at a fixed cadence. The prefill dispatcher uses this data as back-pressure, refusing to send new probes to any prefiler that exceeds a configurable utilization threshold.

D.4 Bit-set subset selection

The Stage-2 subproblem is to choose $Q \subseteq R_{\text{wait}}(k)$ with $|Q| \leq \min(\text{cap}[g], R_{\text{max}})$ that maximizes $F_g(Q)$ in (2). The reduction sketched in Section D.4 expands as follows.

Reduction to reachable sums. $F_g(Q)$ depends on Q only through $\Delta_s(Q) = \sum_{i \in Q} s_i$. Hence, for each cardinality $k \leq R_{\text{max}}$, it suffices to enumerate

$$\mathcal{R}_k := \{\Delta_s(Q) : Q \subseteq R_{\text{wait}}(k), |Q| = k\}.$$

We encode $\mathcal{R}_k$ as a bitmask $\text{dp}[k]$ whose bit b is set iff $b \in \mathcal{R}_k$. The standard 0/1-knapsack recurrence becomes a single shift-OR per item, $\text{dp}[j] \mid = (\text{dp}[j-1] \ll s_i)$, scanning j from k down to 1. Storing dp snapshots at each item allows the chosen subset to be recovered by $O(n)$ backtracking after the optimum is identified.

For BR-0, the score has a single kink and the two-probe rule is exact. For BR-H, the score has up to $H+1$ kinks. In our implementation with $R_{\text{max}} = 4$, we use exhaustive enumeration over the candidate window; this is only $2^{R_{\text{max}}} \leq 16$ subsets per worker and is negligible in the measured dispatch overhead. A bitset implementation can also be made exact by evaluating F over all reachable sums, or by probing around every distinct horizon margin.

Cost. The dominant cost is $O(R_{\text{max}} \cdot S/w)$ word-level operations per worker, where w is the machine word size and S is an upper bound on per-request prompt length used to size the bitmask. This is exponentially smaller than the $O(2^{R_{\text{max}}})$ cost of naive enumeration in R_{max} , while remaining an exact solver under the structure of F_g .

D.5 Engine-side instrumentation for evaluation

For closed-loop validation against the ground-truth straggler dynamics, we apply an optional, lightweight monkey patch to the engine-step loop in vLLM that timestamps every step and records (`step_cost_ms`, `running_reqs`, `total_tokens`, `waiting_reqs`, `kv_cache_usage`) to a per-DP-rank file. The DP rank is recovered from process metadata, with a fallback via launch-argument and PID. The patch is the source of the empirical step traces used in Section 6; it is purely diagnostic and disabled in production runs.

D.6 Pool-bypass deployment path for BR-0

For BR-0 deployments where end-to-end latency is the operator’s priority, the proxy can be configured to bypass the prompt pool. The motivation is structural: in the pool-based path, dispatch waits for a decode slot to free before sending, after which the cross-tier KV transfer is serialized with engine scheduling on the receiving worker. The path described here trades some routing flexibility for an earlier start to that transfer.

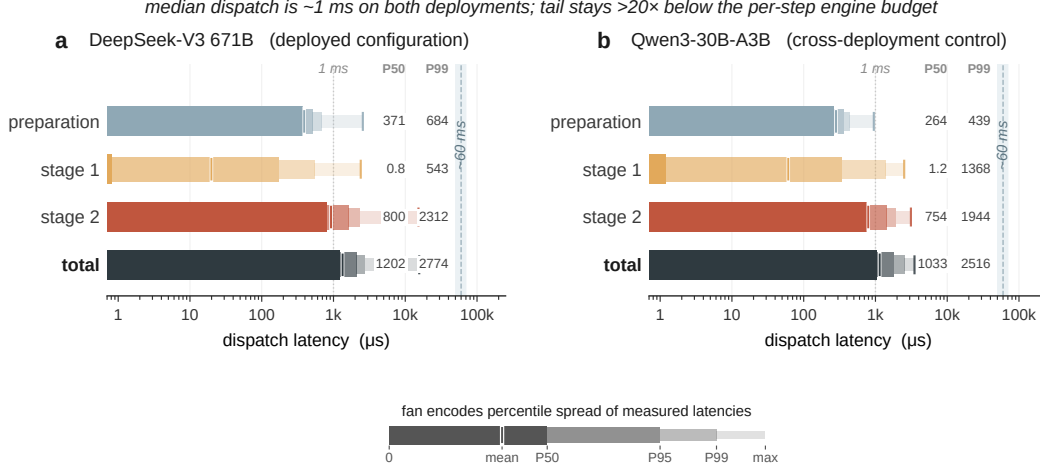


Figure 5: **Per-tick dispatch overhead, two deployments.** $G=8$, $R_{\max}=4$. (a) DeepSeek-V3 671B. (b) Qwen3-30B-A3B (cross-deployment control). Each row is a percentile fan ($0 \rightarrow P_{50}$ solid, then to P_{95} , P_{99} , max); the white tick marks the mean. Shaded band (~ 60 ms) marks the per-step engine latency on DeepSeek-V3.

Mechanism. On prefill completion, the proxy immediately evaluates the BR-0 single-step score $F_g(s) = \alpha s - \beta(s - m_g)_+$ over all decode workers, using a *virtual load* that counts both running and already-dispatched-but-not-yet-running requests on each worker, and forwards the request directly. The target worker enters the receive-pending state immediately, allowing the KV transfer to overlap with the residual compute of currently-running requests rather than starting after slot release. To keep the connector’s send buffers bounded, the dispatcher caps the per-worker inflight to a small fixed margin above the batch limit B ; in our deployment a tight cap was sufficient to keep slots utilized without saturating the buffers at the traffic levels we tested.

Scope. This path is available only for BR-0 ($H=0$): with the pool removed, the dispatcher sees one waiting request at a time, so Stage-2 subset selection collapses to Stage-1 by construction. For BR-H ($H>0$) the pool is required, and the late-binding cost noted in Section 5 applies. A controlled latency benchmark of pool-based BR-0 against the pool-bypass variant, and an extension that preserves Stage-2 leverage while overlapping KV transfer for BR-H, are directions we leave to future work.

D.7 Dispatch overhead

Figure 5 measures the per-tick cost of the BR-H dispatcher itself. We instrument the proxy as described in Appendix D.5 and record per-stage latency on two deployments: the DeepSeek-V3 671B configuration of Section 6.2 (Figure 5a) and a Qwen3-30B-A3B deployment held to identical $G=8$ and $R_{\max}=4$ (Figure 5b).

Total per-loop dispatch cost has $P_{50}=1.20$ ms (mean 1.34 ms, $P_{99}=2.77$ ms) on DeepSeek-V3 and $P_{50}=1.03$ ms (mean 1.15 ms, $P_{99}=2.52$ ms) on Qwen3. The two distributions agree to within 17% at P_{50} and 10% at P_{99} , consistent with dispatcher cost being determined by control-plane work in the proxy rather than engine-side compute. Against the ~ 60 ms per-step engine budget on the deployed configuration (shaded band), this places the dispatcher roughly $50\times$ below the engine step at the median and $\sim 22\times$ below at P_{99} , with a small number of Stage-2 outliers above 10 ms that are still within one engine step.

The per-stage decomposition shows Stage 2 (the bit-set DP) accounting for $\sim 69\%$ of total dispatch time on both deployments (mean 924 μ s on DeepSeek, 794 μ s on Qwen3). Stage 1 is sub-microsecond at the median ($P_{50}=0.8\text{--}1.2$ μ s) but has a longer tail ($P_{99}=0.54$ ms on DeepSeek, 1.37 ms on Qwen3) when the abundant-capacity loop iterates many times before crossing the S_{greedy} threshold. Preparation has the tightest distribution ($P_{99}/P_{50} \approx 1.8$), consistent with its $O(GH)$ structure once G and H are fixed.

E Supplementary trace-level analysis: Survival predictor on Proprietary Data

This appendix supplies the trace-level analysis of BR-H deployed with the empirical-survival predictor on Proprietary Data, the comparison instantiation reported in Table 1. The motivation is to verify that the secondary deployment behaves as the offline characterization in Section C.3 predicts: on this workload empirical-survival has weaker offline signal than the per-prompt memorization variant, so the corresponding deployed router should do worse end-to-end—but still substantially better than the baselines and better than BR-0, because the Stage-1 confidence gate at $\hat{p}_{\text{fin}} \geq 0.5$ closes cleanly when the predictor lacks information rather than injecting noise (Section C.2.3).

Trace-level worker spread. Figure 6 plots the per-worker instantaneous KV-cache footprint over the same 1,500-step segment used in Figure 3, with the BR-H router driven by the empirical-survival predictor. The trace-mean imbalance is 40,757 tokens, against 36,277 for ExactMatch (Figure 3, bottom-right), 51,927 for BR-0, and 215,110 for the strongest baseline JSQ. Visually, the band of eight worker curves remains tightly clustered throughout the trace; the spread is $\sim 1.1\times$ that of ExactMatch and $\sim 1.3\times$ tighter than BR-0, consistent with the predictor’s role of adding short-horizon resolution to BR-0’s prediction-free baseline even when its Stage-1 separation on this workload is weak.

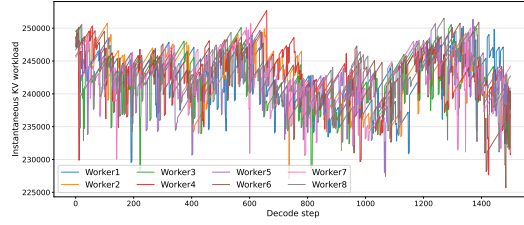


Figure 6: **Per-worker instantaneous KV-cache workload on Proprietary Data under BR-H deployed with the empirical-survival predictor**, $H=80$, $G=8$ decode workers, same 1,500-step trace segment as Figure 3. Trace-mean imbalance is 40,757 tokens. The band remains visibly tighter than BR-0 (51,927) and far tighter than the strongest baseline JSQ (215,110); it is somewhat wider than the ExactMatch deployment (36,277). The Stage-1 confidence gate (Section C.2.3) closes for the long-tail majority of requests on this workload, so $\hat{c}_i(k)$ falls back to the conservative anchor H rather than injecting a noisy point estimate; the gain over BR-0 comes from the requests for which the predictor is confident.

Headline numbers in context. On every metric reported in Table 1, BR-H with empirical-survival improves over BR-0: imbalance reduces by 21.5% (51,927 $\rightarrow$ 40,757), TPOT P95 reduces by 0.7% (79.32 $\rightarrow$ 78.75 ms), and throughput rises by 3.9% (942.5 $\rightarrow$ 979.2 tok/s). Compared to JSQ, the same router achieves a $5.3\times$ imbalance reduction and a 16.2% throughput improvement. Compared to its better-aligned counterpart ExactMatch, the empirical-survival deployment is $\sim 12\%$ behind on imbalance (40,757 vs. 36,277) and essentially tied on throughput.

Reading this in context. The Survival deployment on Proprietary Data is the failure-mode case for the predictor: the workload offers no auxiliary signal beyond the marginal CDF, and the marginal CDF itself is heavy-tailed, so the predictor’s output is conservative. The empirical observation is that the BR-H rule degrades gracefully under this regime—the gate closes, $\hat{c}_i(k)$ collapses to H , and the router behaves like a horizon-anchored variant of BR-0 rather than an oracle approximation. This is the design property targeted by the Stage-1 confidence gate: the predictor abstains cleanly on requests it cannot resolve, so a worse predictor yields a degraded but still-useful router rather than a router that would perform worse than the prediction-free baseline.

F Per-worker KV-workload panels

This appendix provides the per-worker KV-cache workload panels that were omitted from Figures 3 for space and the panels for Azure-2024. Layout matches the corresponding main-text figure: each

panel plots all eight workers on the same 1,500-step decode segment, with the trace-mean imbalance annotated under each panel.

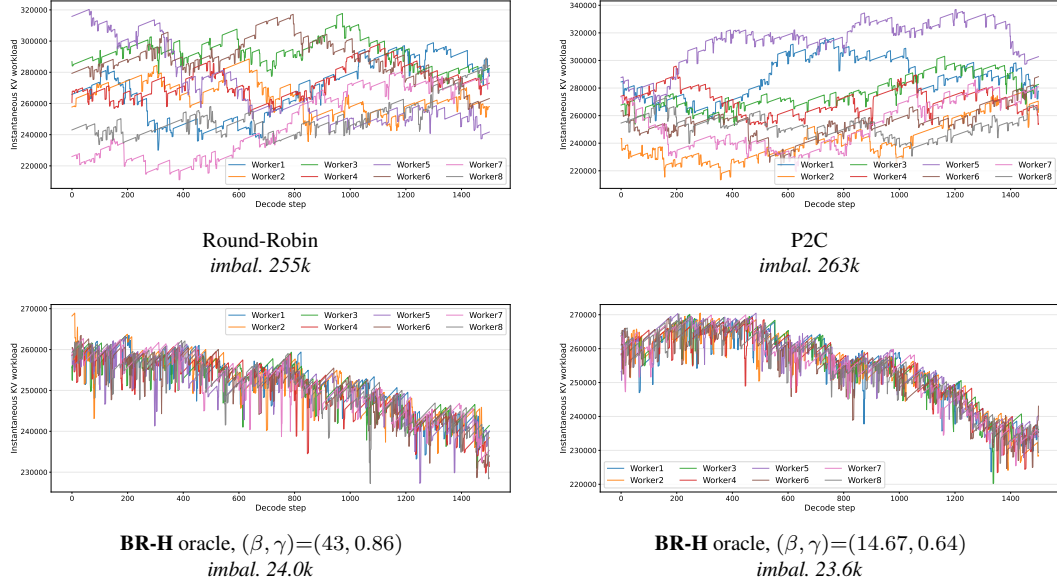


Figure 7: **Remaining per-worker panels for Proprietary Data, $G=8$** (companion to Figure 3). Top: RR and P2C baselines. Bottom: two BR-H oracle operating points.

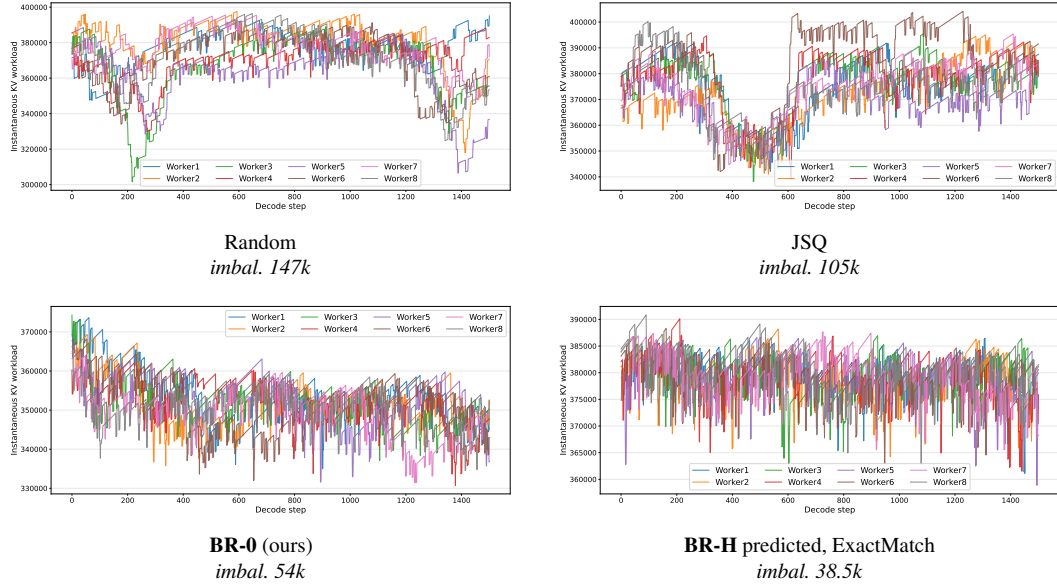


Figure 8: **Per-worker KV-cache workload on Azure-2024, $G=8$** . Same layout as Figure 3.

G Trace-level evidence for the scaling sweep at $G \in \{4, 16\}$

This appendix complements Section 6.3 with per-cluster-size headline numbers (Tables 4 and 5) and per-worker KV-workload trace galleries (Figures 10 and 11) for the two cluster sizes outside the $G=8$ main configuration: $G=4$ on 2P1D and $G=16$ on 5P1D. The trace counterpart at $G=8$ is Figure 3 in the main text. Each G 's table and figure are co-located here so that the appendix is self-contained; the same numbers also appear in compressed form in Table 2 of the main text.

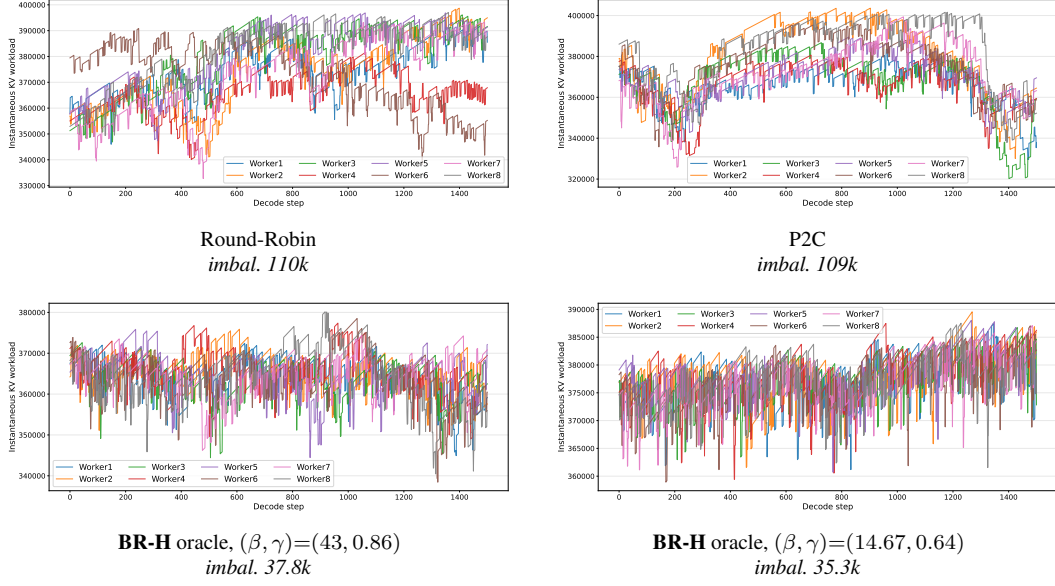


Figure 9: **Remaining per-worker panels for Azure-2024, $G=8$ (companion to Figure 8).** Top: RR and P2C baselines. Bottom: two BR-H oracle operating points.

On the choice of oracle prediction. Section 6.3 reports BR-H with oracle prediction at every G in order to isolate the router’s scaling behaviour from any confounding effect of predictor quality. The deployed (predictor-driven) BR-H is characterised against its oracle counterpart at $G=8$ in Section 6.2 on the main Proprietary Data configuration; the per-worker traces at $G=4$ and $G=16$ shown below are the corresponding oracle traces only, since we did not run the deployed predictor at the other two cluster sizes. The widening of the BR-H advantage with G reported in Section 6.3 is a structural consequence of the order-statistics gap (Section 2.1) and is captured by the router itself; quantifying how predictor noise interacts with G is left to future work.

Headline numbers and traces at $G=4$. Table 4 reports the average imbalance, TPOT P95, and throughput of every method at the smallest cluster size in the sweep ($G=4$ on 2P1D, 48 NPUs); Figure 10 shows the corresponding per-worker KV-cache traces. The four standard baselines fall in the 64–80k imbalance range and around 365 tok/s throughput; BR-0 reaches 9.7k imbalance and 407 tok/s; BR-H with oracle prediction at $H=80$ reaches 6.6k imbalance and 415 tok/s, the lowest TPOT P95 in the table.

Table 4: **All methods on Proprietary Data at $G=4$ (2P1D, 48 NPUs).** Reproduces the $G=4$ column of Table 2 as a self-contained companion to Figure 10. Best in each column in **bold**.

Method	Avg. imbalance ↓	TPOT P95 (ms) ↓	Throughput (tok/s) ↑
Random	80,118	66.30	364.7
Round-Robin	68,108	65.27	365.7
P2C	64,012	65.40	366.4
JSQ	75,326	66.45	365.4
BR-0	9,711	63.19	407.1
BR-H oracle, $H=80$	6,576	62.52	415.3

Headline numbers and traces at $G=16$. Table 5 reports the analogous numbers at the largest cluster size in the sweep ($G=16$ on 5P1D, 144 NPUs), including both oracle BR-H operating points used in the main paper; Figure 11 shows the corresponding traces. The four baselines fall in the 616–688k imbalance range and around 926 tok/s throughput; BR-0 reaches 215k imbalance and 1,118 tok/s; the two BR-H oracle operating points reach 117–124k imbalance and 1,247–1,251 tok/s, with $(\beta, \gamma)=(14.67, 0.64)$ achieving the lowest imbalance and lowest TPOT P95, and $(43, 0.86)$

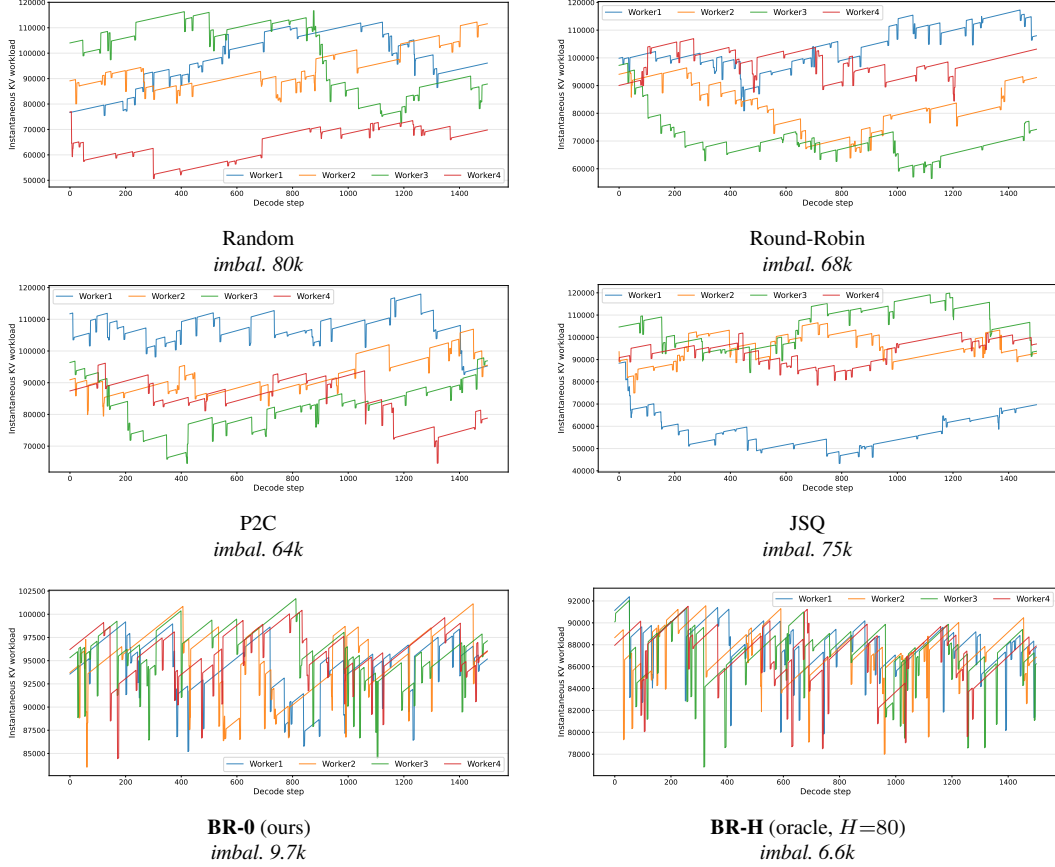


Figure 10: **Per-worker KV-cache traces on Proprietary Data at $G=4$ (2P1D, 48 NPUs).** The four DP workers’ KV-cache footprints over a 1,500-step decode segment under each method; trace-mean imbalance under each panel matches the $G=4$ column of Table 2.

marginally higher throughput by ~ 4 tok/s. We use $(14.67, 0.64)$ as the representative BR-H point in the main-text Figure 4.

Table 5: **All methods on Proprietary Data at $G=16$ (5P1D, 144 NPUs).** Reproduces the $G=16$ column of Table 2 as a self-contained companion to Figure 11, with both oracle BR-H operating points reported. Best in each column in **bold**.

Method	Avg. imbalance $\downarrow$	TPOT P95 (ms) $\downarrow$	Throughput (tok/s) $\uparrow$
Random	638,062	77.80	923.6
Round-Robin	616,089	77.91	925.3
P2C	688,283	77.46	927.3
JSQ	675,607	77.40	926.6
BR-0	215,170	74.92	1,117.8
BR-H <i>oracle</i> , (43, 0.86)	123,700	72.15	1,251.1
BR-H <i>oracle</i> , (14.67, 0.64)	117,067	72.04	1,247.4

This appendix complements Section 6.4 with per-worker KV-workload traces under each swept (β, γ) configuration on Proprietary Data at the two cluster sizes for which we ran a sweep: $G=8$ (4P1D, the main configuration) and $G=16$ (5P1D, the largest configuration). Each sub-panel of the trace galleries plots all G DP workers’ instantaneous KV-cache footprints simultaneously over the same 1,500-step decode segment used in Section 6.2; the vertical spread of the curves is the instantaneous imbalance.

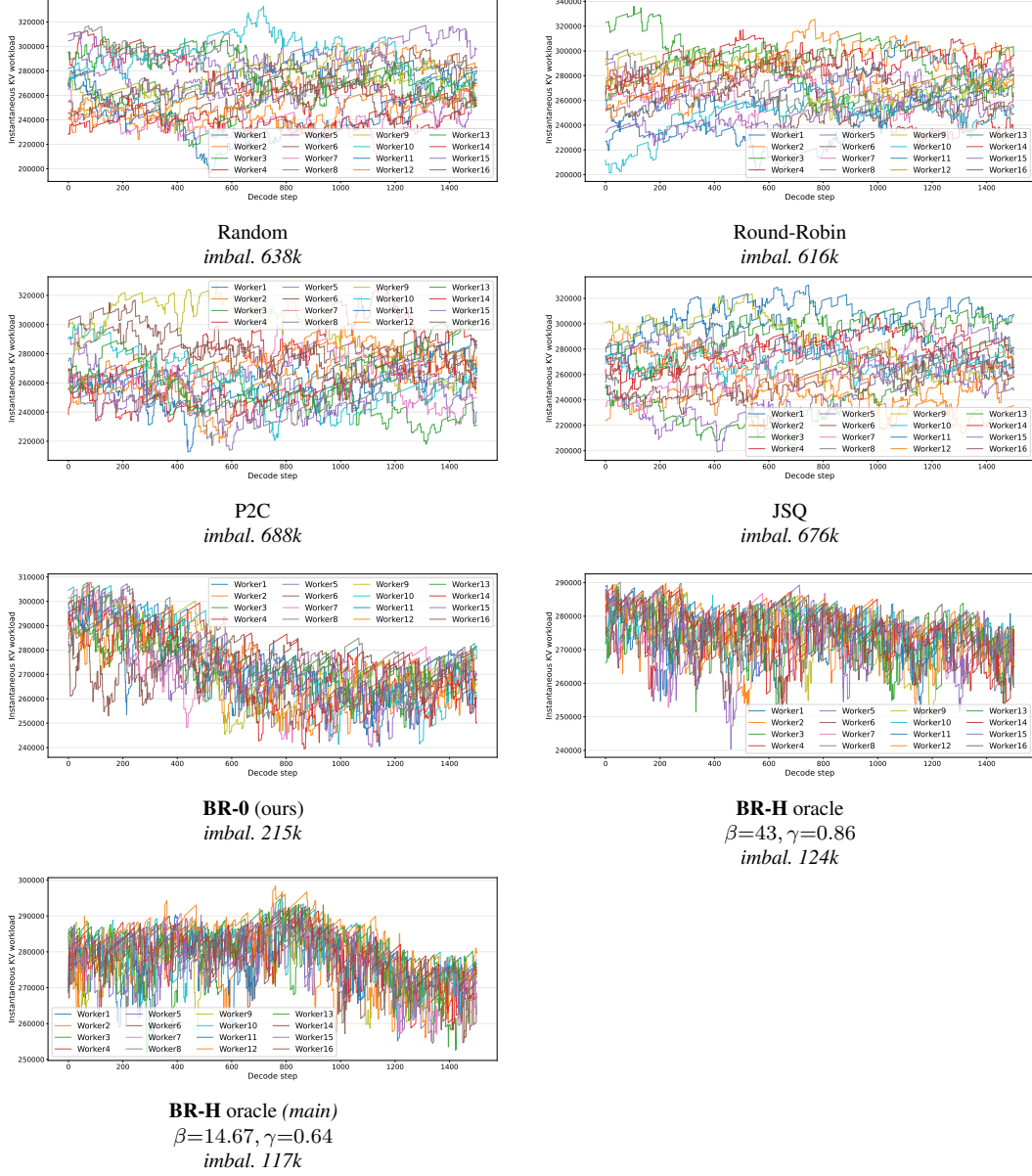


Figure 11: **Per-worker KV-cache traces on Proprietary Data at $G=16$ (5P1D, 144 NPUs).** Layout matches Figure 10; trace-mean imbalance under each panel matches the $G=16$ column of Table 2. Both oracle BR-H operating points are shown for direct comparison; $(\beta, \gamma)=(14.67, 0.64)$ is the configuration plotted in the main-text Figure 4.

H Trace-level evidence for the sensitivity sweep at $G \in \{8, 16\}$

Sensitivity at $G=8$. Table 6 provides the cross-shaped sweep around $(\beta=48, \gamma=0.9)$; we display it here for reference next to its trace gallery. The sweep covers $\beta \in \{1, 24, 48, 96\}$ at $\gamma=0.9$ and $\gamma \in \{0.5, 0.7, 0.9, 1.0\}$ at $\beta=48$. Trace-mean imbalance ranges from 24.7k to 32.3k tokens across the swept region; throughput from 945 to 1,042 tok/s; TPOT P95 from 77.3 to 79.5 ms. By comparison, every standard baseline at $G=8$ on Proprietary Data (Table 1) sits at 215–438k imbalance and 810–847 tok/s throughput—an order-of-magnitude imbalance gap and a > 100 tok/s throughput gap. Every swept configuration sits well inside the high-throughput, low-imbalance regime.

Table 6: (β, γ) **sensitivity sweep at $G=8$ (4P1D, 96 NPUs) on Proprietary Data, BR-H oracle, $H=80$.** Cross-shaped sweep around $(\beta=48, \gamma=0.9)$.

Sweep	β	γ	Avg. imbalance ↓	TPOT P95 (ms) ↓	Throughput (tok/s) ↑
β at $\gamma=0.9$	1	0.9	32,304	78.120	986.41
	24	0.9	24,706	77.933	1,022.67
	48	0.9	25,608	77.517	1,042.61
	96	0.9	31,862	79.460	945.41
γ at $\beta=48$	48	0.5	28,327	78.040	1,024.04
	48	0.7	26,599	77.611	1,020.78
	48	0.9	25,608	77.517	1,042.61
	48	1.0	29,318	77.310	1,026.77

Figure 13 reports BR-H sensitivity to (β, γ) in (2) on Proprietary Data under oracle prediction at $H=80$. We sweep the parameter plane along two axes intersecting at $(\beta=48, \gamma=0.9)$: $\beta \in \{1, 24, 48, 96\}$ at fixed $\gamma=0.9$, and $\gamma \in \{0.5, 0.7, 0.9, 1.0\}$ at fixed $\beta=48$, giving a cross-shaped sweep of seven configurations.

Sensitivity at $G=16$. Table 7 reports the analogous cross-shaped sweep at $G=16$ (5P1D), again under BR-H oracle prediction with $H=80$. The same seven (β, γ) operating points are evaluated. Trace-mean imbalance ranges from 104k to 127k tokens across the swept region; throughput from 1,243 to 1,257 tok/s; TPOT P95 from 71.1 to 73.2 ms. The relative spread of imbalance (factor $\sim 1.2\times$ across the sweep) and throughput (factor $\sim 1.01\times$ across the sweep) is even tighter than at $G=8$, indicating that the robustness pattern observed at $G=8$ persists at $G=16$. By comparison, every standard baseline at $G=16$ on Proprietary Data sits at 616–688k imbalance and 924–927 tok/s throughput (Table 2); the BR-H sweep is 5–6 $\times$ tighter on imbalance and $\sim 34\%$ higher on throughput than every baseline at this scale.

Table 7: (β, γ) **sensitivity sweep at $G=16$ (5P1D, 144 NPUs) on Proprietary Data, BR-H oracle, $H=80$.** Same cross-shaped sweep as Table 6, evaluated at the largest cluster size.

Sweep	β	γ	Avg. imbalance ↓	TPOT P95 (ms) ↓	Throughput (tok/s) ↑
β at $\gamma=0.9$	1	0.9	104,229	71.583	1,256.30
	24	0.9	117,879	72.198	1,248.51
	48	0.9	118,396	72.610	1,255.77
	96	0.9	112,771	72.260	1,256.17
γ at $\beta=48$	48	0.5	116,074	72.227	1,247.32
	48	0.7	109,763	71.114	1,256.53
	48	0.9	118,396	72.610	1,255.77
	48	1.0	127,207	73.167	1,243.79

Reading the $G=16$ gallery. The qualitative pattern matches the $G=8$ gallery: every panel shows a tightly clustered 16-worker band with no visibly diverging worker, and the band shape is similar across the (β, γ) region. Visually, the bands at $G=16$ are wider in absolute terms than at $G=8$ —consistent with the order-statistics scaling discussed in Section 6.3—but the spread *within* the swept region remains a small fraction of the gap to any baseline panel of Figure 11, the trace counterpart at the same scale. The robustness conclusion of Section 6.4 therefore extends to the largest cluster size we tested: BR-H is not a knife-edge configuration even at $G=16$.

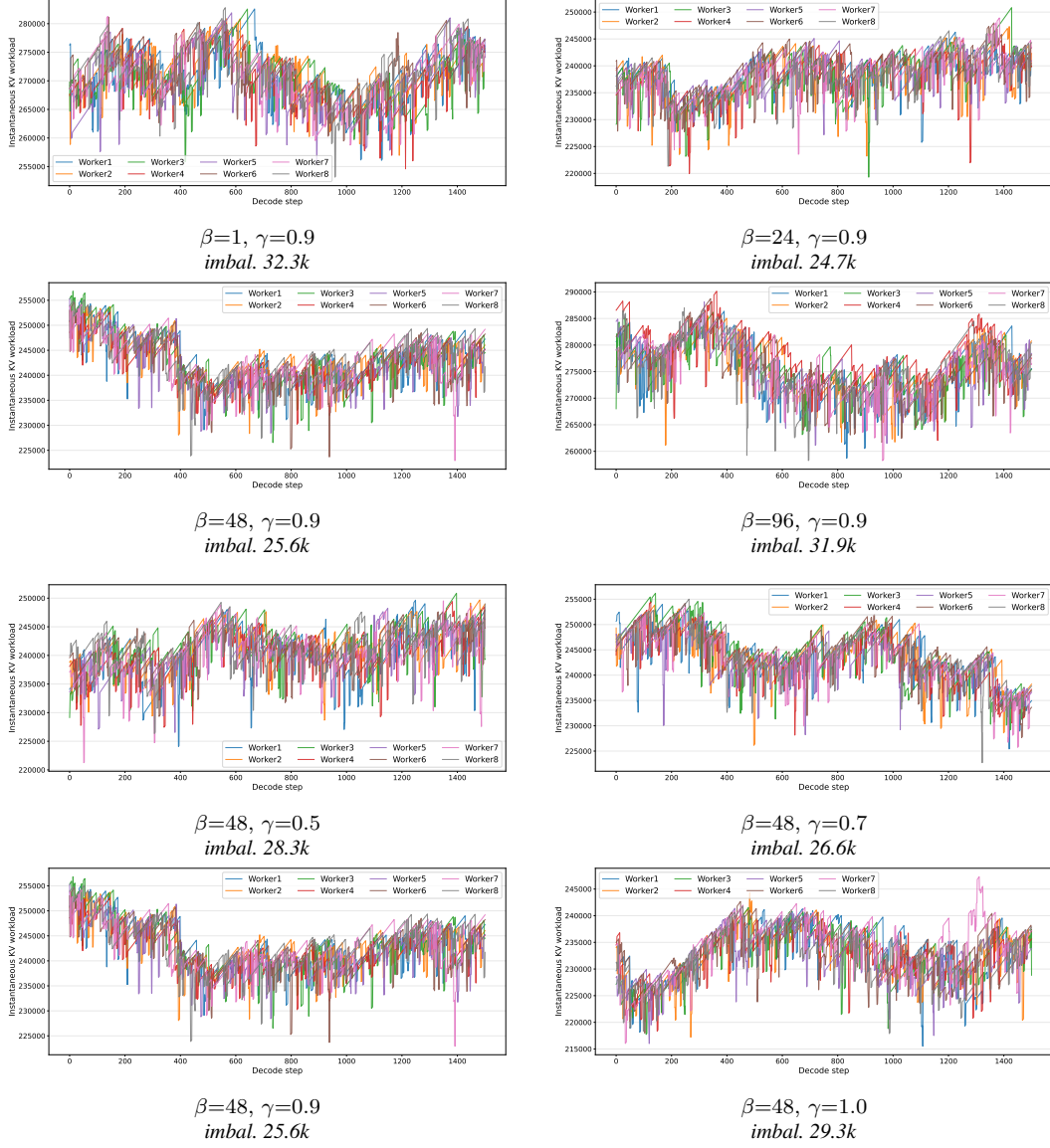


Figure 12: **Per-worker KV-cache traces under each swept (β, γ) on Proprietary Data at $G=8$, BR-H oracle, $H=80$.** Each panel is annotated with its (β, γ) and trace-mean imbalance. The configuration $(\beta=48, \gamma=0.9)$ is shared between the β - and γ -sweep blocks and therefore appears in both. Per-panel y-axes are independently autoscaled.

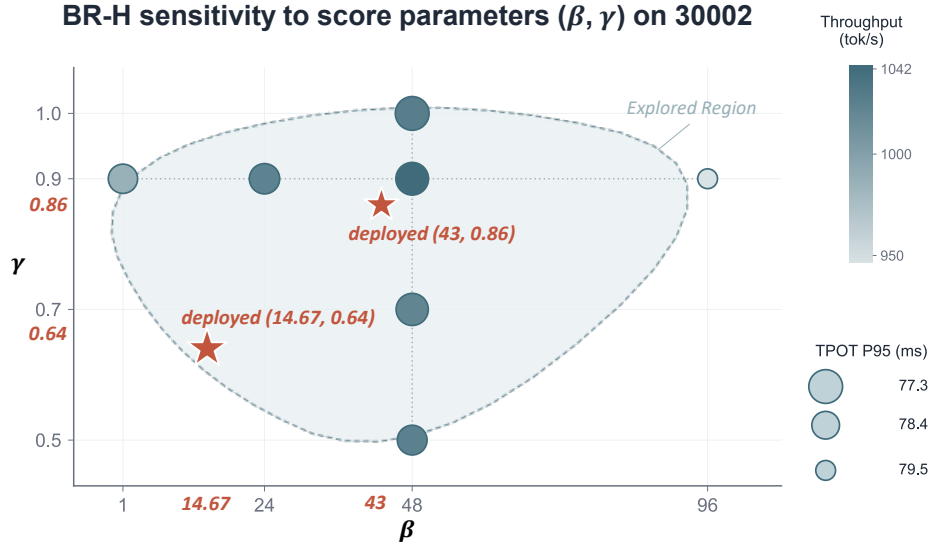


Figure 13: **BR-H sensitivity to (β, γ) on Proprietary Data, oracle, $H=80$.** Marker color: throughput (darker = higher). Marker size: $1/\text{TPOT P95}$ (larger = faster). Circles: 7 swept configurations. Stars: 2 deployed configurations.

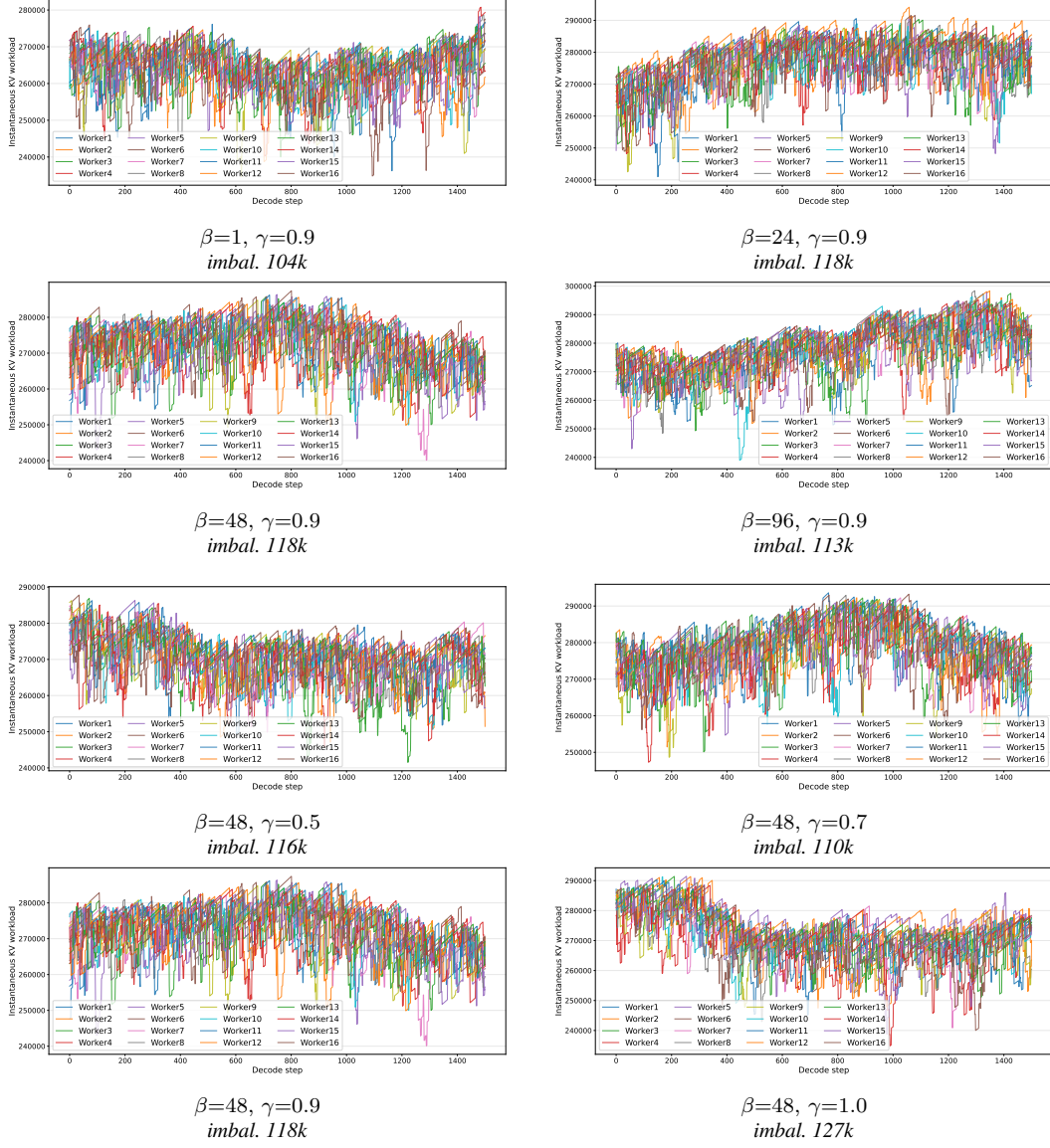


Figure 14: **Per-worker KV-cache traces under each swept (β, γ) on Proprietary Data at $G=16$, BR-H oracle, $H=80$.** Same convention as Figure 12; each panel is labelled by (β, γ) and trace-mean imbalance.